ITE3162 – WEB DEVELOPMENT

Unit 4

Dr. Eric Hitimana

Outlines

1. **Recall Case Study – Gathering Requirements**
2. **Introduction to WEB SERVICES**
3. **HTTP Protocol**
4. **Presentations (3groups)**
5. Introduction to Django Framework
6. Django Hands-on
7. Django REST Framework
8. Django API Hands-on

1. Requirement Gathering Before Coding

Understand what to build, and for whom, before deciding how to build it

1 — Identify stakeholders

Who will use, pay for, or run the system?

2 — Collect needs

Interviews, questionnaires, observation, existing documents

3 — Classify

Functional vs non-functional requirements

4 — Model

User stories, use cases, wireframes

5 — Document & validate

Requirements document reviewed and approved by the client

Case example: *Smart Seeds Nurserying and Distribution Management, Karongi District*

Functional: what the system does

Nurseries record seedling stock; farmers request seedlings; officers approve and track distribution.

Non-functional: how well it does it

Works on phones with weak internet, offers Kinyarwanda and English, restricts access by user role.

Only then: design and code

Clear requirements prevent rework; every feature you code should trace back to one.

1.1 TOPIC: Smart Seeds Nurserying and Distribution Management, Case of Karongi District

---

- **Group Members:**
  1. **2012444**
  2. **2021393**
  3. **2039233**

a. Problem

- Lack of ready seeds localization
- Seeds category distribution
- Lack of seeds company categorization per seed type
- Monitoring of seed consumers for future decision making
- Waste of water for irrigation practice
- Lack of Seeds with fertilization type mapping

b. Components & Features

Responsive WEB APP

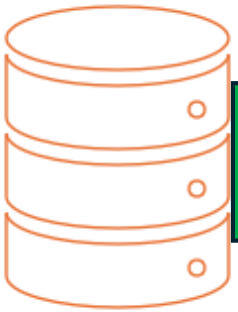
- User authentication & Authorization
- Localization
- User role Management
- Cooperative Management
- Plot/Farm management
- Seed category Management
- Seed Management
- Localization management
- Fertilizer Category Management
- Fertilizer Management
- *Device (IoT) Management*
- *Device Allocation vs status Management*
- Seed Production *Forecasting*
- Data Visualization & Report

Mobile APP

- Authentication
- Monitor the farm
- Access ready seeds with details
- Book ready seeds
- Access irrigation schedule using AI
- Offline mode capability

IoT Device

- ✓ Power management
- ✓ Data acquisition
- ✓ Data submission



DB

Configuration

External System

c. Users

- **Individual Farmer**
- **Cooperative Manager**
- **Local government leaders**
- **Partners (NAEB)**
- **Admin**



- **Web Service**s play a crucial role in modern software development by enabling *seamless integration* between *disparate systems*.

Responsive WEB APP

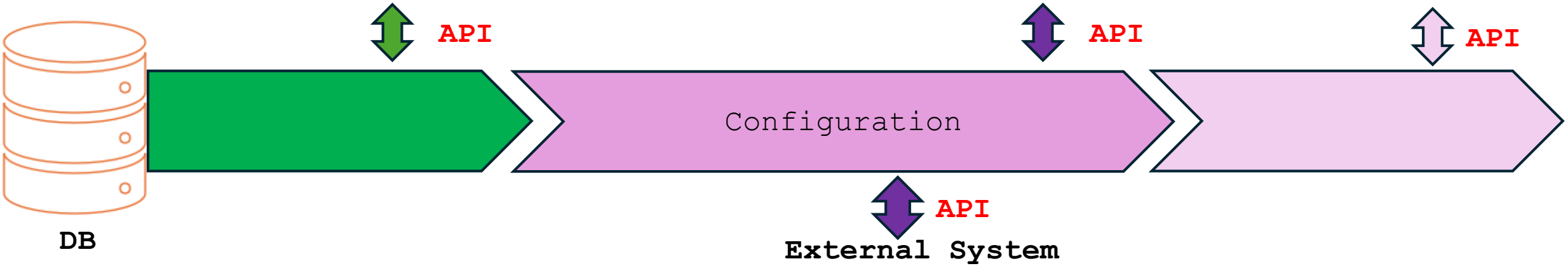
- User authentication & Authorization
- Localization
- User role Management
- Cooperative Management
- Plot/Farm management
- Seed category Management
- Seed Management
- Localization management
- Fertilizer Category Management
- Fertilizer Management
- Device (IoT) Management
- Device Allocation vs status Management
- Data Reporting Management
- Data Visualization Management

Mobile APP

- Authentication
- Monitor the farm
- Access ready seeds with details
- Book ready seeds
- Access irrigation schedule using AI
- Offline mode capability

IoT Device

- ✓ Power management
- ✓ Data acquisition
- ✓ Data submission



2.1 Introduction to APIs and Web Services

How software applications talk to each other

API

Application Programming Interface

- A set of rules and protocols that lets software applications communicate, share data, and request operations from each other.
- Acts as a bridge: hides internal complexity and offers a standard way to interact.

Web Service

An API accessed over the web

- A type of API that works over a network using HTTP/HTTPS.
- Lets applications on different platforms exchange data in standard formats such as JSON or XML.

Common approaches:

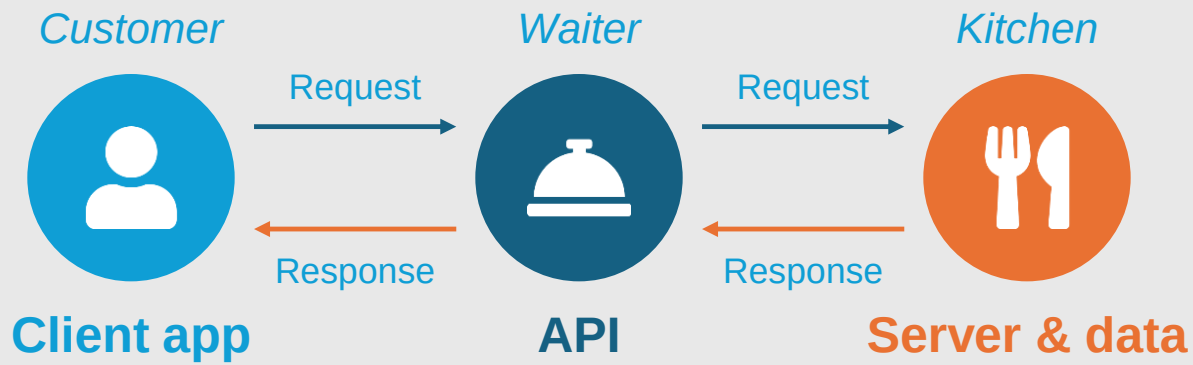
SOAP protocol | **REST** architectural style | **XML-RPC** protocol

Key idea: *every web service is an API, but not every API is a web service.*

2.2 The API Concept

An API is the go-between that carries requests and responses

- **Intermediary:** lets one application use data or services from another system.
- **Rulebook:** defines what can be requested, how to ask, and what comes back.
- **Integration:** connects services and data across different platforms and technologies.



You order through the waiter and never enter the kitchen; an app calls the API and never touches the server's internals.

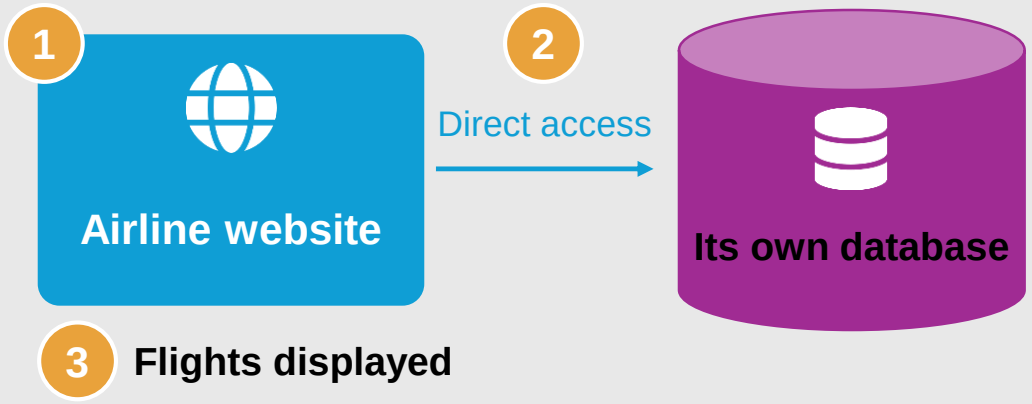
Where APIs are used

- E-commerce
- Social media
- Home automation
- Network programming

2.3 API Example: Travel Booking

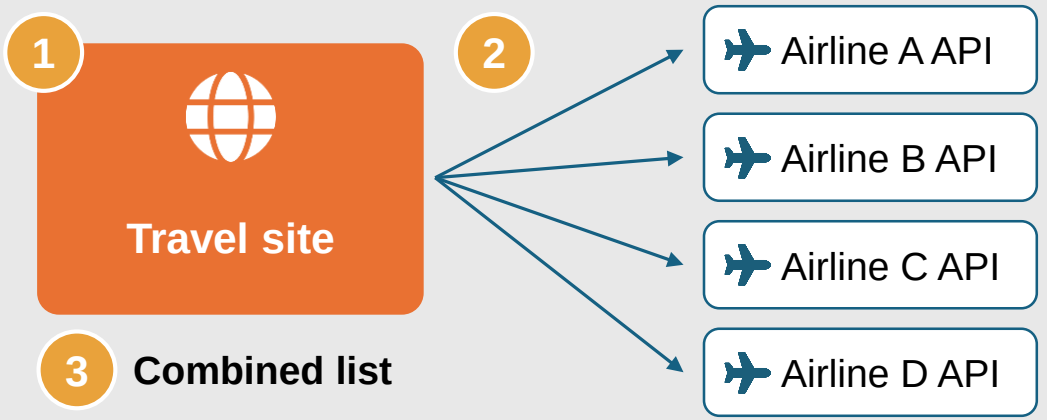
One search, many airlines: APIs let a single site gather flights from every carrier

Airline website: one source



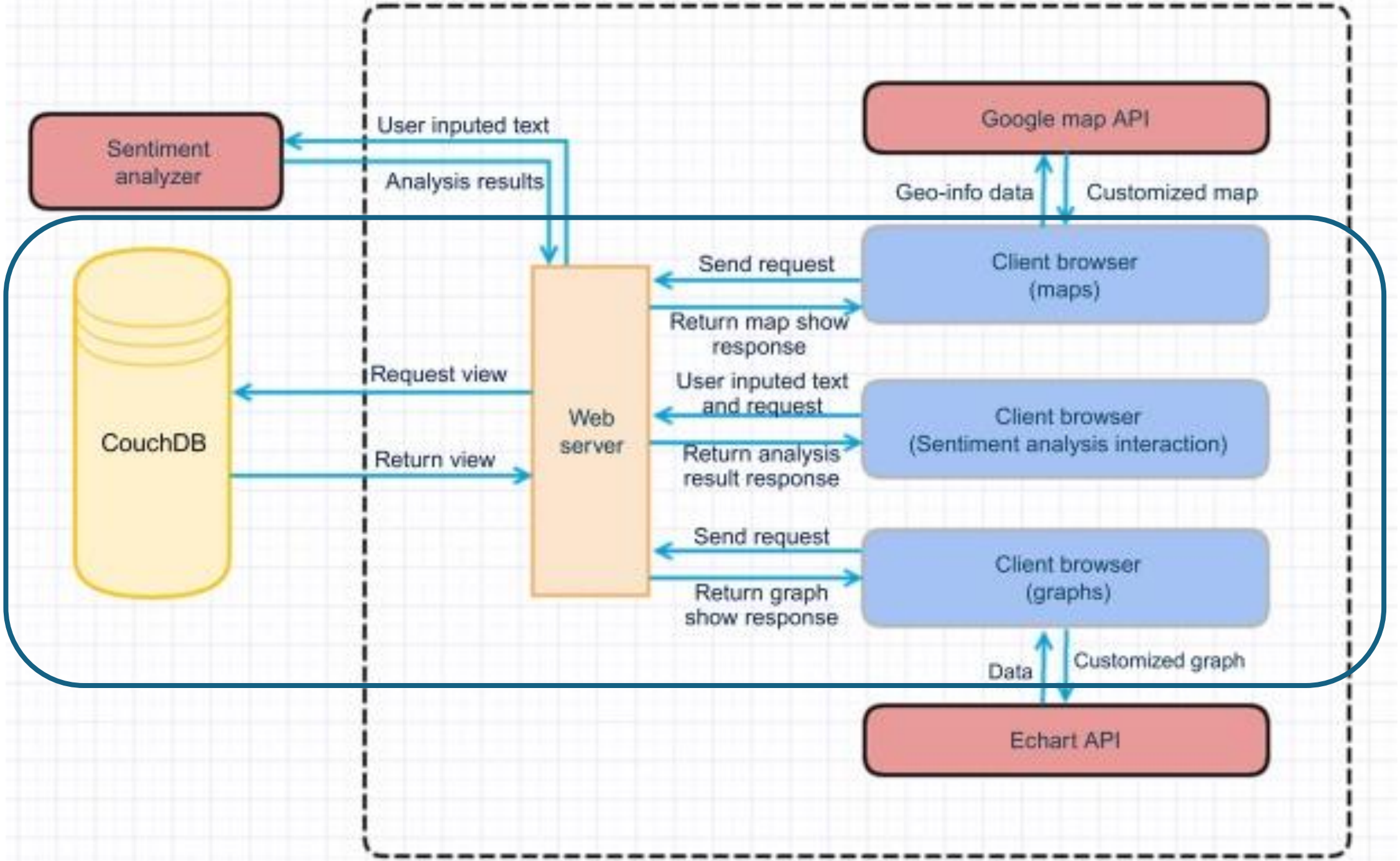
You see only that one airline's flights.

Travel website: many sources via APIs



You compare flights from many airlines at once.

1. You enter your trip details once
2. The site calls each airline's API at the same time
3. Results are combined into one list to compare


Sentiment analyzer
User inputed text
Analysis results
Google map API
Geo-info data
Customized map
Client browser (maps)
Send request
Return map show response
CouchDB
Request view
Return view
Web server
User inputed text and request
Return analysis result response
Client browser (Sentiment analysis interaction)
Send request
Return graph show response
Client browser (graphs)
Data
Customized graph
Echart API

2.4 The Evolution of Distributed APIs

From remote procedure calls to web APIs, 1970s to today



Dawn of distributed computing

1970s – 1980s

1976 **RPC**
Proposed in RFC 707

1984 **RPC in practice**
Birrell & Nelson system

1980s **Message queues**
Asynchronous messaging

Object-oriented APIs

1990s

1991 **CORBA**
Objects across languages

1993 **COM / DCOM**
Microsoft; DCOM in 1996

1997 **Java RMI**
Remote Java objects

Web-based APIs

Late 1990s – today

1998 **XML-RPC**
Calls as XML over HTTP

1999 **XMLHttpRequest**
Browsers call APIs

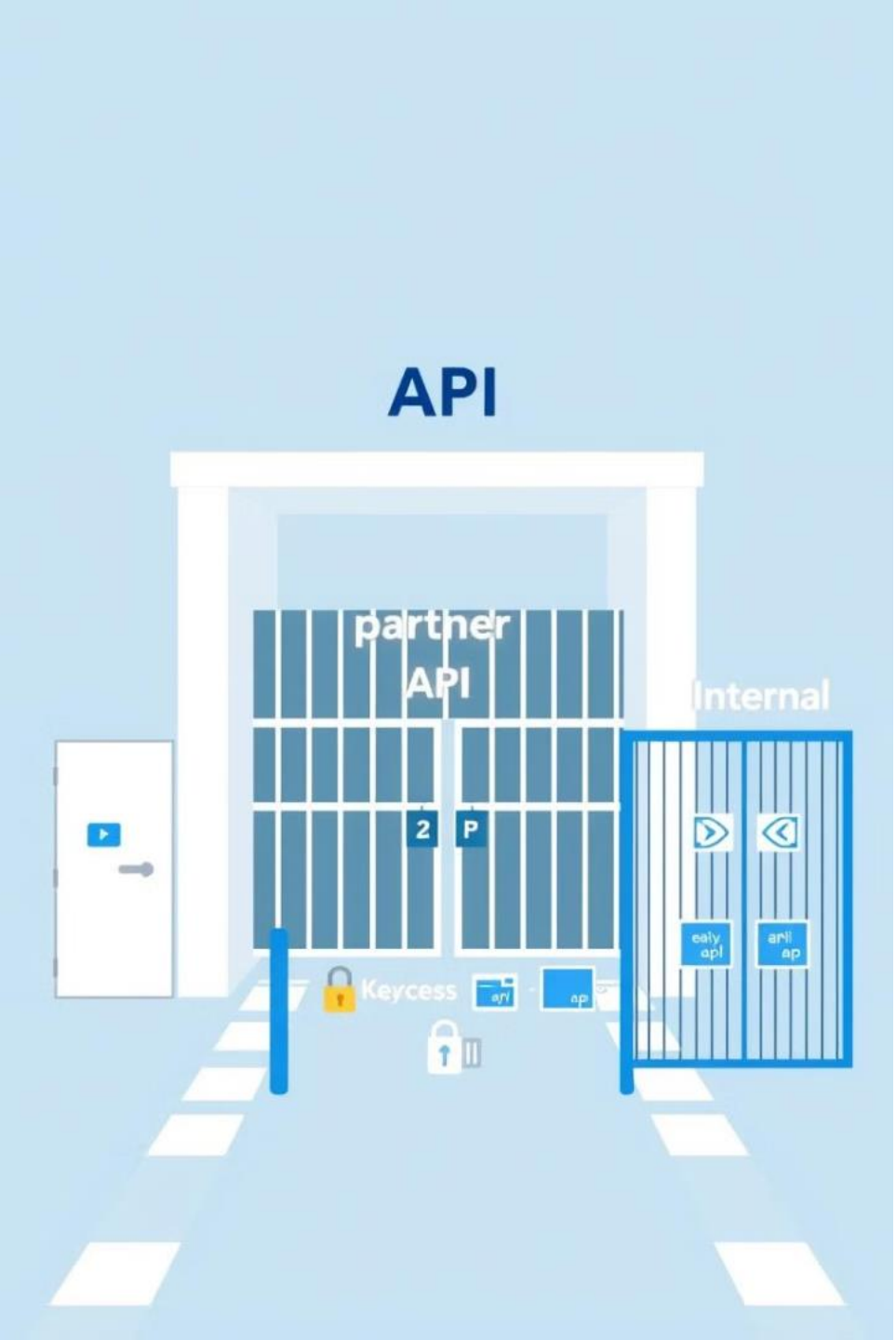
2000 **SOAP 1.1**
XML messaging (W3C)

2000 **REST**
Architectural style (Fielding)

2015 **GraphQL**
Client picks the data it needs

2015 **gRPC**
Google; fast calls over HTTP/2

Sources: White, RFC 707 (1976); Birrell & Nelson, ACM TOCS (1984); OMG, CORBA 1.0 (1991); W3C, SOAP 1.1 Note (2000); Fielding, dissertation (2000).



2.5 Types of APIs

Open/Public APIs

- ❑ Publicly available with no restrictions. May require a free key for usage control.

Internal/Private APIs

- ❑ Used within an organization for internal data and service access.

Partner APIs

- ❑ Used between a company and its business partners, requiring specific permissions.

2.6 Purpose of Web Services

1 **Integration**

- ❑ Facilitate the integration of disparate systems within and across organizations.

2 **Interoperability**

- ❑ Enable communication between applications built on different technologies and platforms.

3 **Reusability**

- ❑ Promote reuse of software components, reducing development time and effort.

4 **Scalability**

- ❑ Support on-demand access and utilization of services as business needs evolve.



2.6 Purpose of Web Services

5 Service-Oriented Architecture (SOA)

- Web services are a fundamental building block of service-oriented architecture. They provide the means to encapsulate and expose business functionality as services, to be used by other applications as needed.

6 Cross-platform communication

- Web services enable applications running on different platforms and operating systems to communicate with each other seamlessly.

7 Industry Standardization

- Adhere to standardized protocols and formats, ensuring compatibility and fostering collaboration within and across industries.



2.7 Types of Web Services

RESTful-Based

- Uses HTTP principles and often utilizes JSON or XML for data exchange.

SOAP-Based

- Uses XML for message exchange and relies on WSDL (Web Services Description Language) for service definition.

XML-RPC-Based

- Simple and lightweight protocol that uses XML messages for remote procedure calls.
- It allows applications to run on different platforms and invokes methods on remote systems.

JSON-RPC-Based

- JSON-RPC is a lightweight protocol like XML-RPC but uses JSON for message serialization instead of XML. It enables remote procedure calls between applications that can communicate using JSON.

GraphQL-Based

- GraphQL is a query language and runtime for APIs. It allows clients to request and receive only the specific data they need, reducing over-fetching and under-fetching of data.

OData-Based

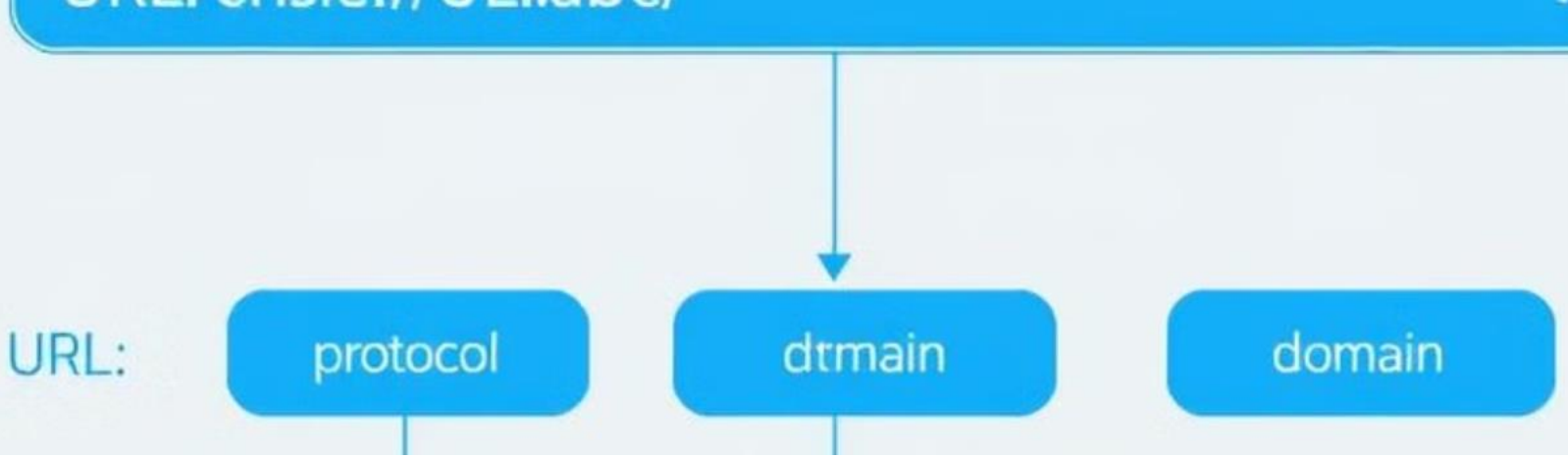
- Odata (Open Data Protocol) is a protocol that enables the creation and consumption of RESTful APIs for data sources.

2.8 REST and RESTful APIs

- **REST** (Representational State Transfer) is an **architectural style** for designing web service applications. It works on top of the HTTP protocol, defining functions for requests and responses.
- **RESTful APIs** use common HTTP methods (POST, GET, PUT, PATCH, DELETE) corresponding to CRUD operations (Create, Read, Update, Delete).

1. **Client Request**
   Client sends HTTP request for data
2. **Server Processing**
   Server processes request according to API rules
3. **Server Response**
   The server sends back data, often in JSON format
4. **Client Handling**
   Client application uses received data





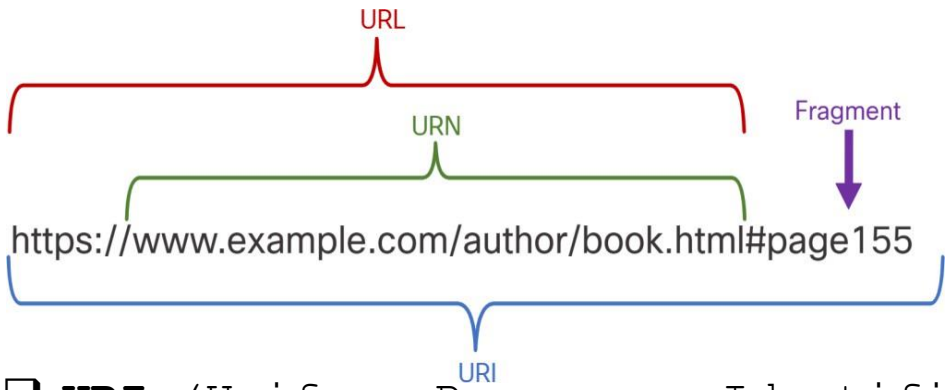
3 HTTP and URI Components

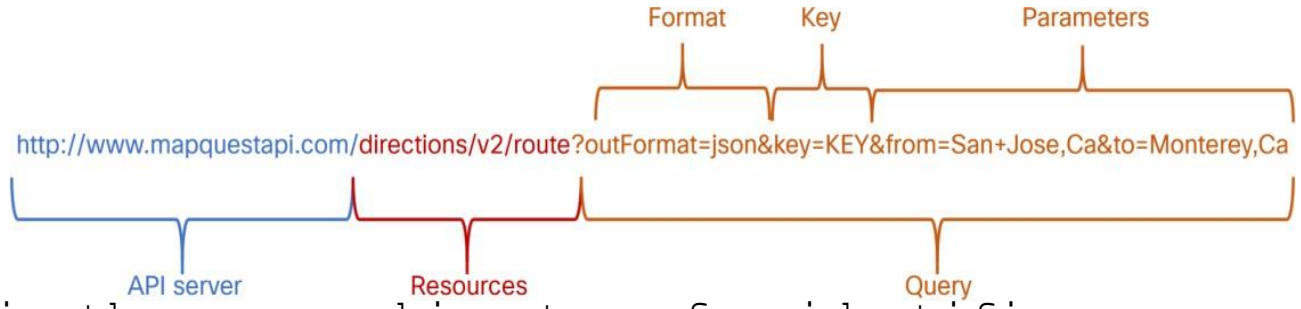
- **HTTP** (Hypertext Transfer Protocol) is fundamental for web communication, enabling data exchange between clients and servers.
- **URIs** (Uniform Resource Identifiers) identify web resources, with URLs specifying network locations and URNs identifying namespaces.

1	2	3	4
Protocol/Scheme	**Hostname**	**Path and Filename**	**Fragment**
HTTPS, FTP, etc.	www.example.com	/author/book.html	#page155

3.1 HTTP and URI Components - Example

1	2	3	4
Protocol/Scheme	Hostname	Path and Filename	Fragment
HTTPS, FTP, etc.	www.example.com	/author/book.html	#page155





- **URI** (Uniform Resources Identifier) is the overarching term for identifiers.
- **URL** (Uniform Resource Locator) is a locator, showing where to access a resource.
- **URN** (Uniform Resource Name) is a name, typically stable over time and independent of location

3.2 HTTP: The Foundation of Web Communication

- Hypertext Transfer Protocol (HTTP) is the cornerstone of data exchange on the World Wide Web.
- The fundamental protocol enabling seamless communication between clients and servers, facilitating the retrieval of web resources, and powering modern web applications and services.
- Its simplicity, extensibility, and widespread adoption have made HTTP an indispensable part of our digital infrastructure.



3.3 Key Features of HTTP

1 Client-Server Model

- HTTP operates on a client-server model, where clients (typically web browsers) send requests to servers, which respond with requested resources or actions.

2 Statelessness

- Each HTTP request is independent, allowing for simple request-response interactions without server-side session state maintenance.

3 Request Methods

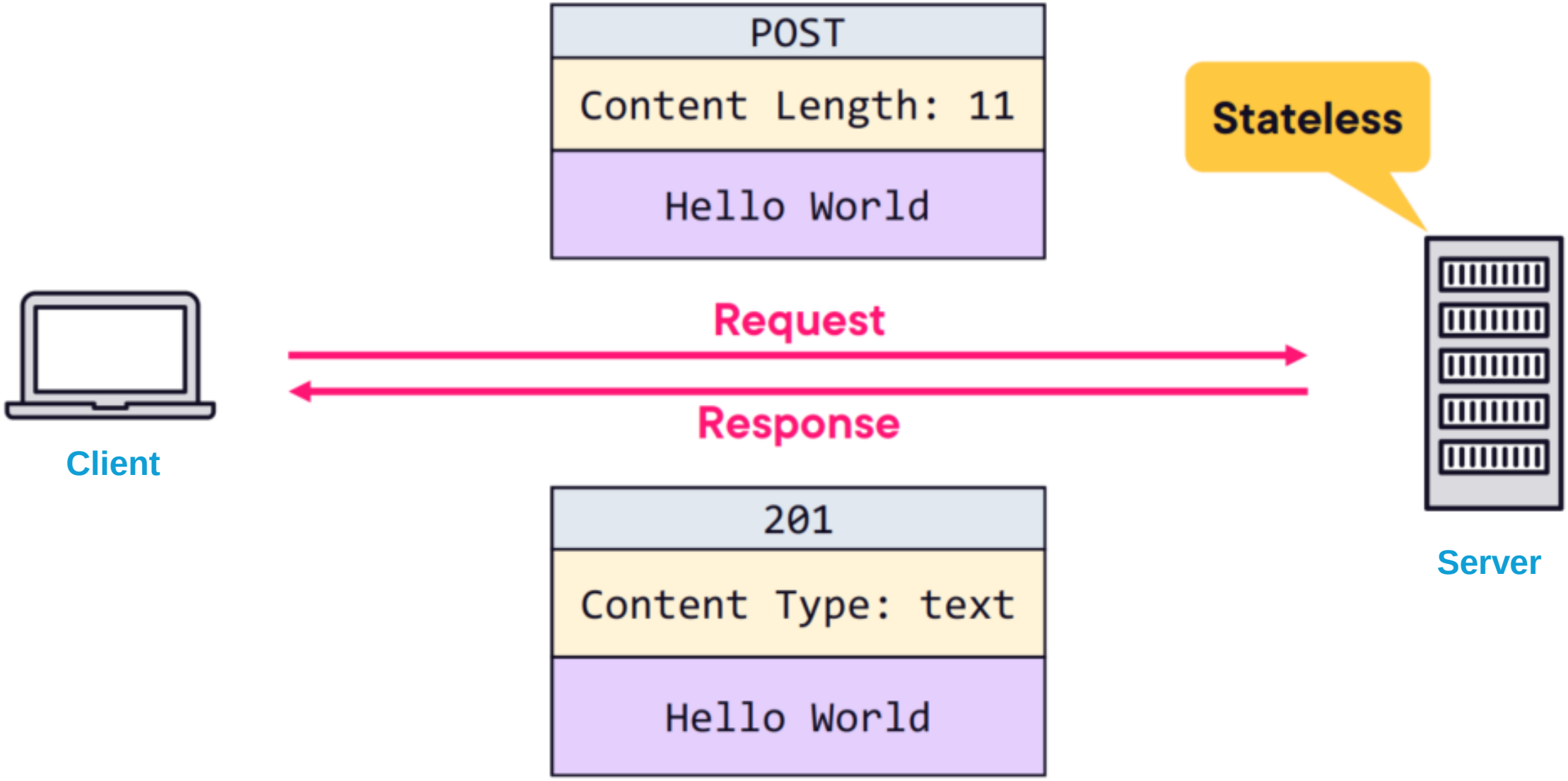
- HTTP defines various request methods (GET, POST, PUT, DELETE) to indicate desired actions on resources.

4 Uniform Resource Identifier (URI)

- URIs are used to identify and locate resources on the web, consisting of a scheme, domain name, and path.



3.4 How Does HTTP Works?



HTTP Verbs/Methods

GET

- Retrieves a resource from the server.

POST

- Submits data to be processed by the server. Used for form submissions.

PUT

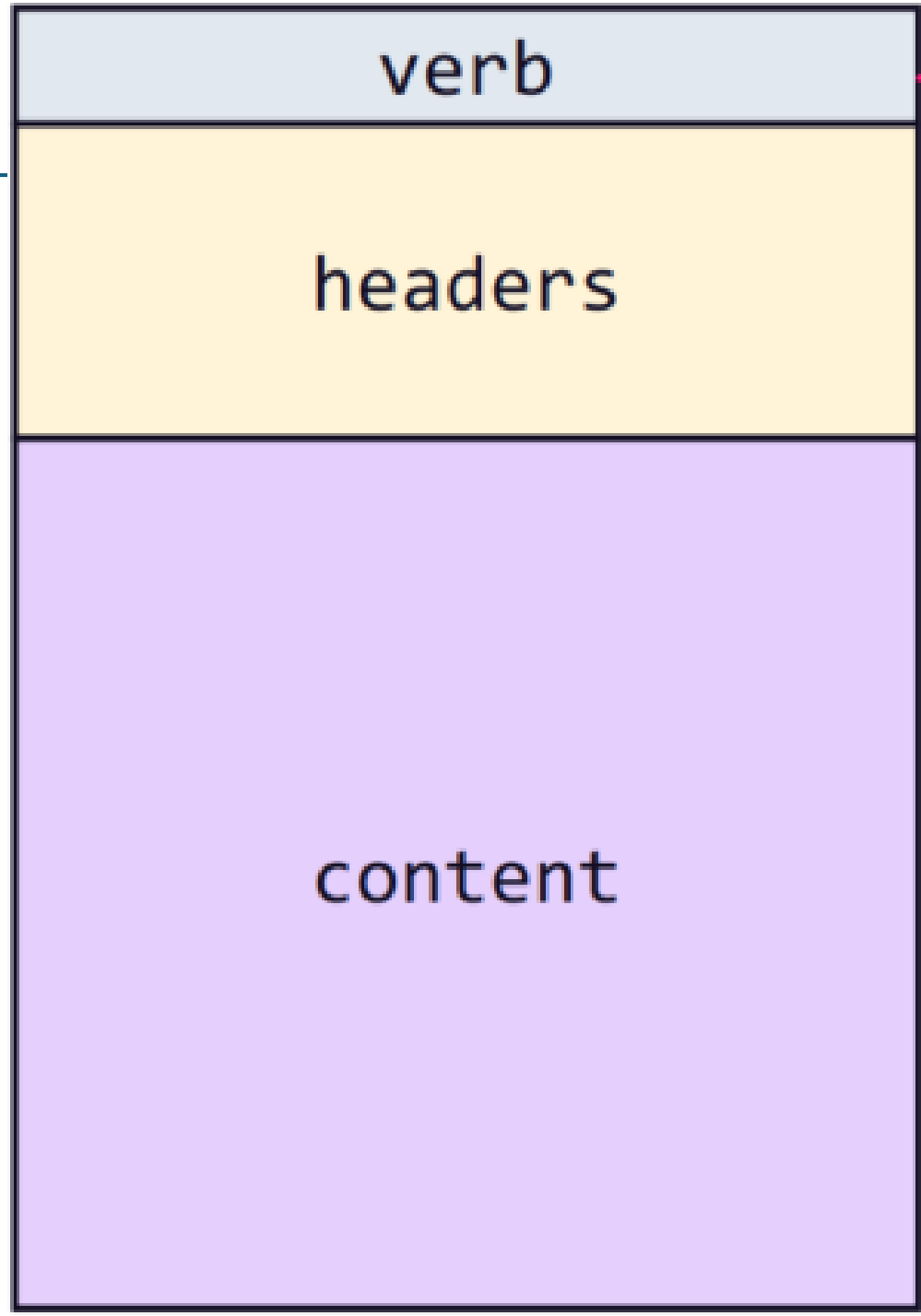
- Updates an existing resource on the server.

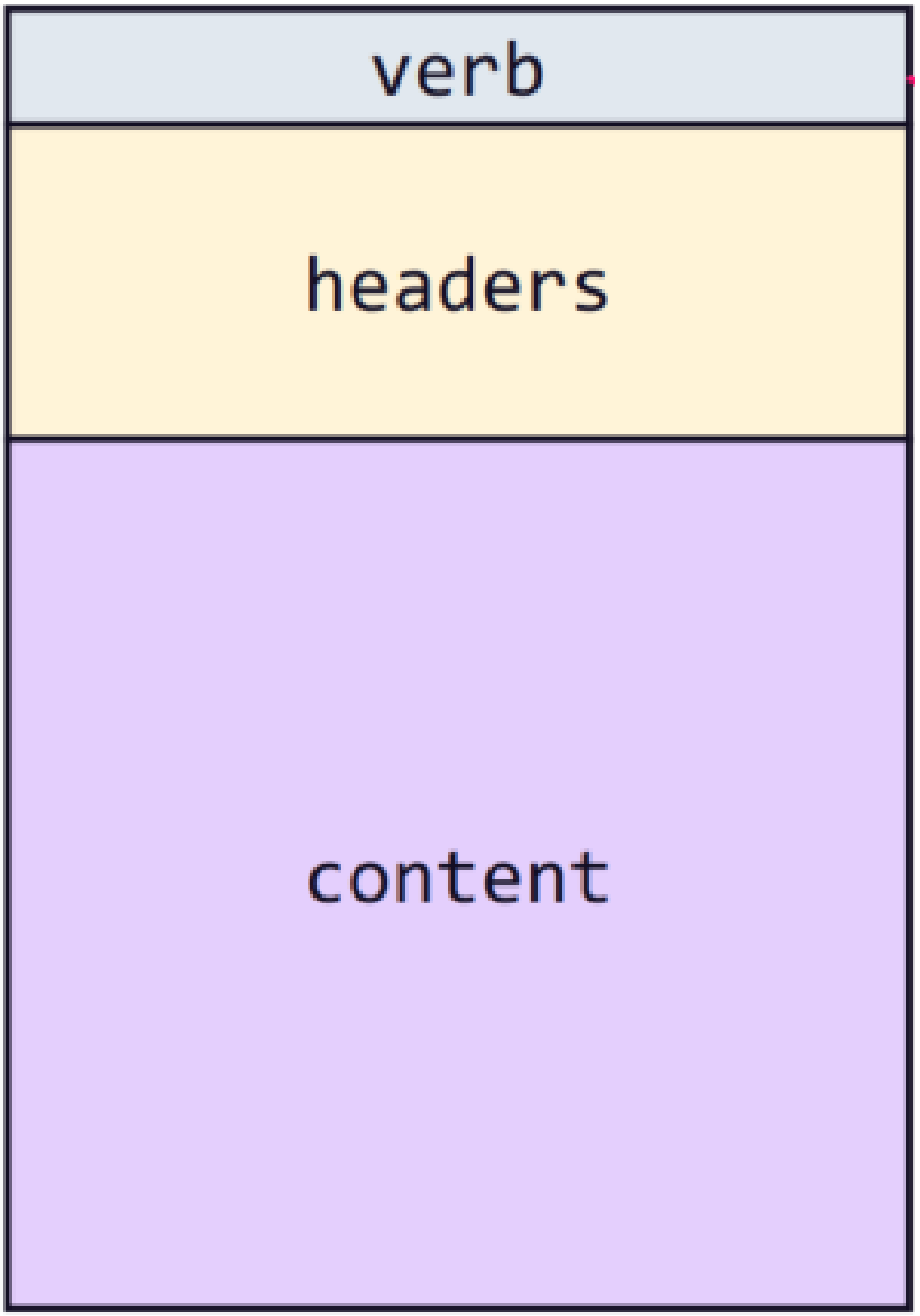
PATCH

- Updates partial resources.

DELETE

- Removes a resource from the server.

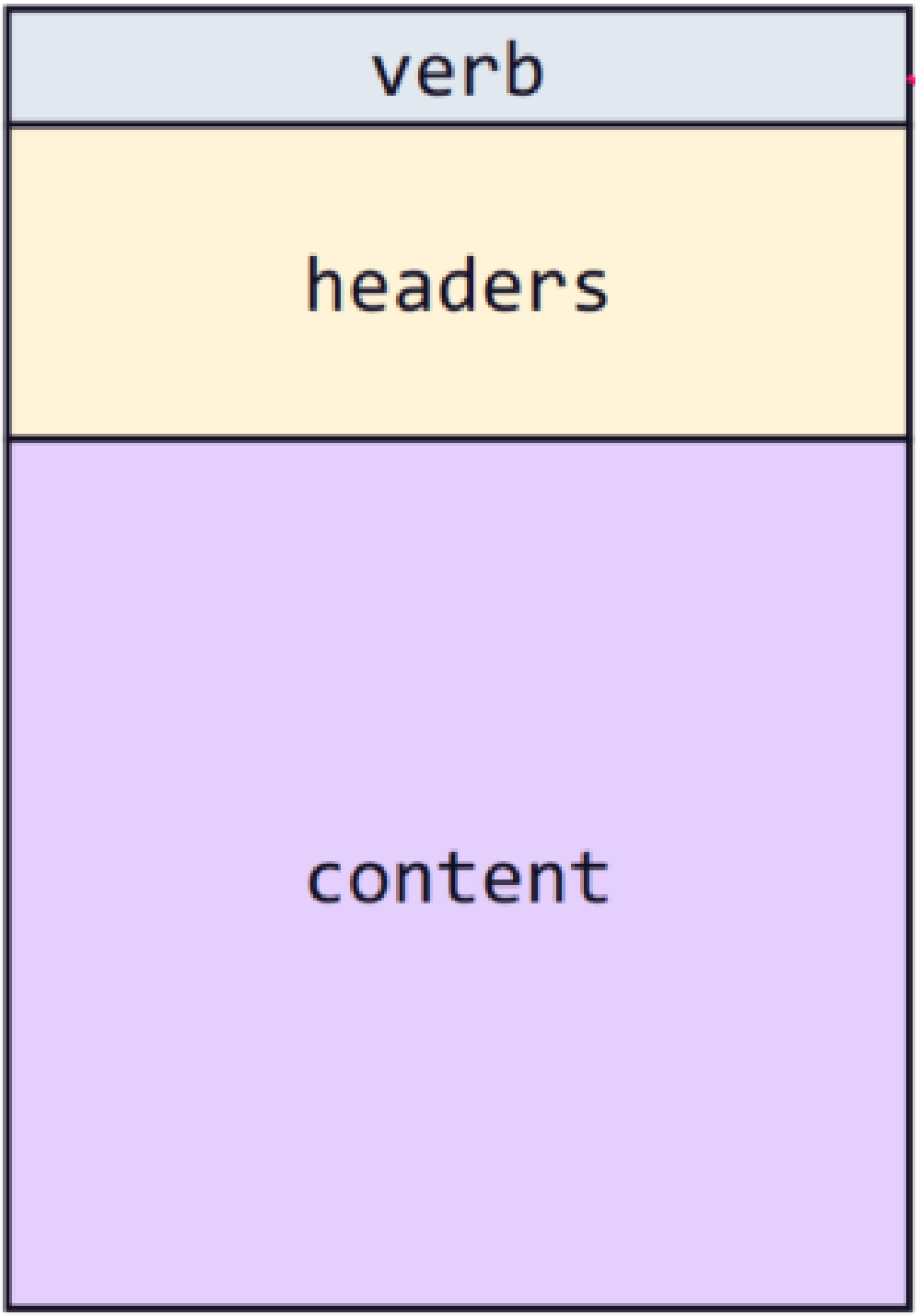




HTTP Headers

Metadata about the request

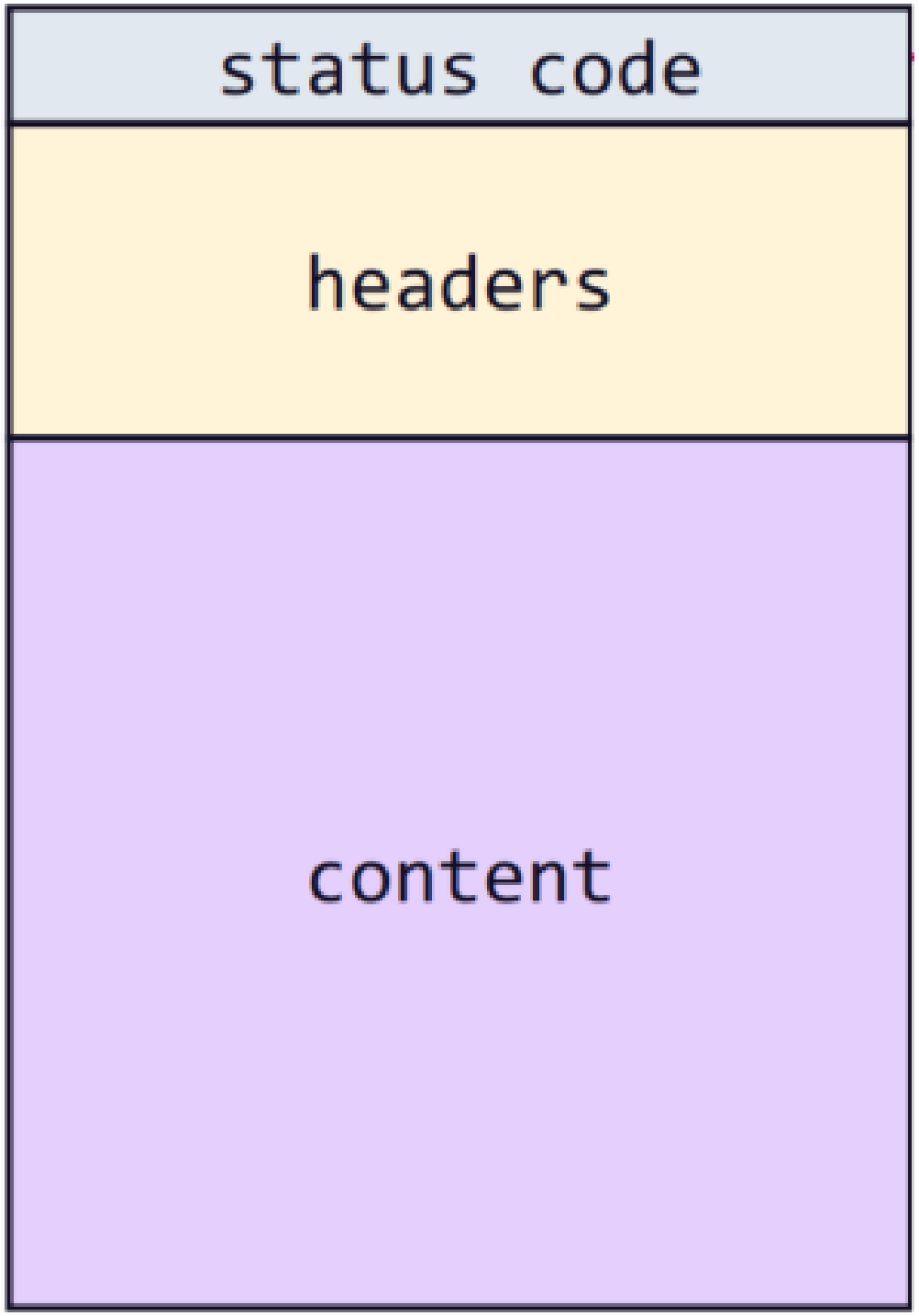
- Content-Type: The format of content
- Content-Length: Size of content
- Authorization: Who's making the call
- Accept: What type(s) can accept
- Cookies: Client data in the request
- More headers...



Content

Data to send to server

- HTML, CSS, JavaScript, XML, JSON
- Content is not valid with some verbs
- Information to help fulfill request
- Binary and blobs common (e.g. .jpg)



HTTP Status Codes

1xx	Informational	100 Continue, 101 Switching Protocols
2xx	Success	200 OK, 201 Created, 204 No Content
3xx	Redirection	301 Moved Permanently, 302 Found, 304 Not Modified
4xx	Client Errors	400 Bad Request, 401 Unauthorized, 404 Not Found
5xx	Server Errors	500 Internal Server Error, 503 Service Unavailable

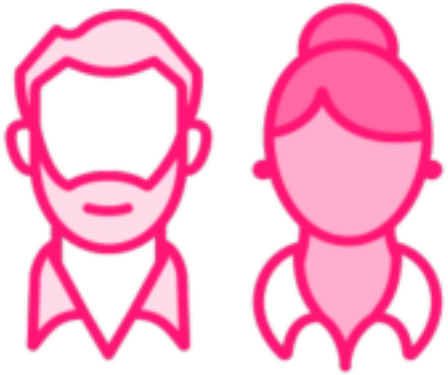
REST API – More with example

What are URIs?

- URIs are just paths to resources
  - E.g. api.yourserver.com/people
- Query strings for non-data elements
  - E.g. format, sorting, searching, etc.

REST API

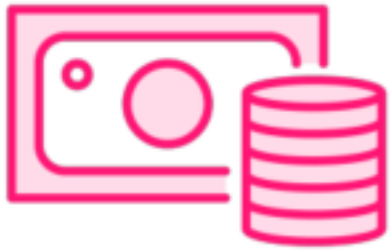
❑ **What are Resources?**



People



Invoices



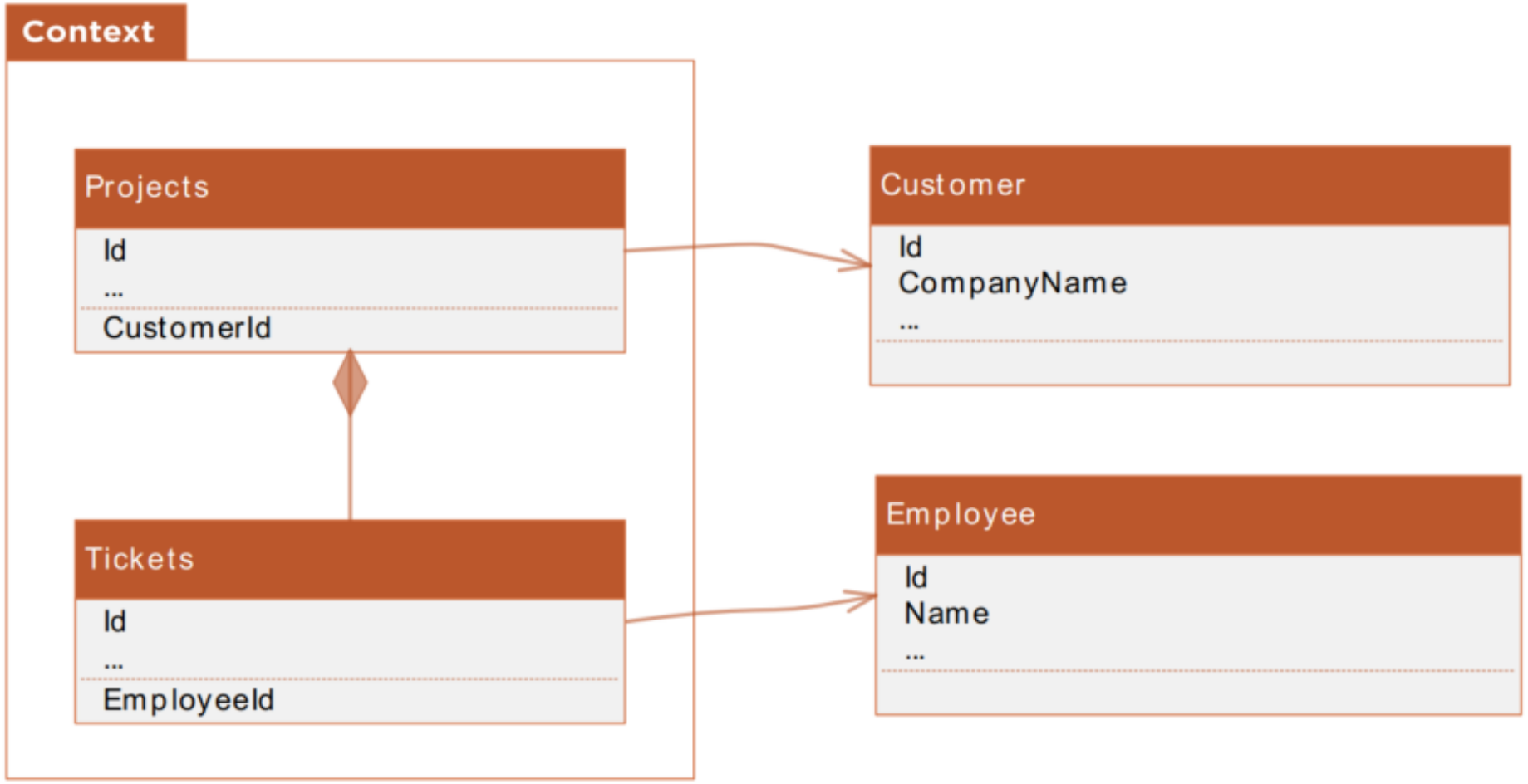
Payments



Products

REST API - Resources

❑ **What are Resources?**



REST API – Identifier in URIs

Identifiers in URIs

- Use unique identifiers
- Does not have to be 'primary keys'

```
/customers
/customers/1
/customers/microsoft
/customers/msft
...
```

REST API – Query Strings

Query strings

- Use for non-resource properties

```
/customers?includeProjects=true
/tickets?page=1
...
```

REST API – Verbs & URIs

How do I do verbs then?

Resource	GET (read)	POST (create)	PUT (update)	DELETE (delete)
/customers	Get list	Create item	Update batch	Error
/customers/123	Get item	Error	Update item	Delete item

3.5 HTTP Versions

1. HTTP/1.1

   The original widely-used version introduced persistent connections and additional methods.

2. HTTP/2

   Introduced multiplexing and compression techniques for improved performance.

3. HTTP/3

   Designed to improve performance over unreliable network connections using the QUIC (Quick UDP Internet Connection) transport protocol.





Further Learning Resources

Specifications

HTTP/1.1 Specification: RFC 2616

HTTP/2 Specification: RFC 7540

Developer Resources

Mozilla Developer Network: MDN Web Docs - HTTP

Google Developers: Web Fundamentals - HTTP

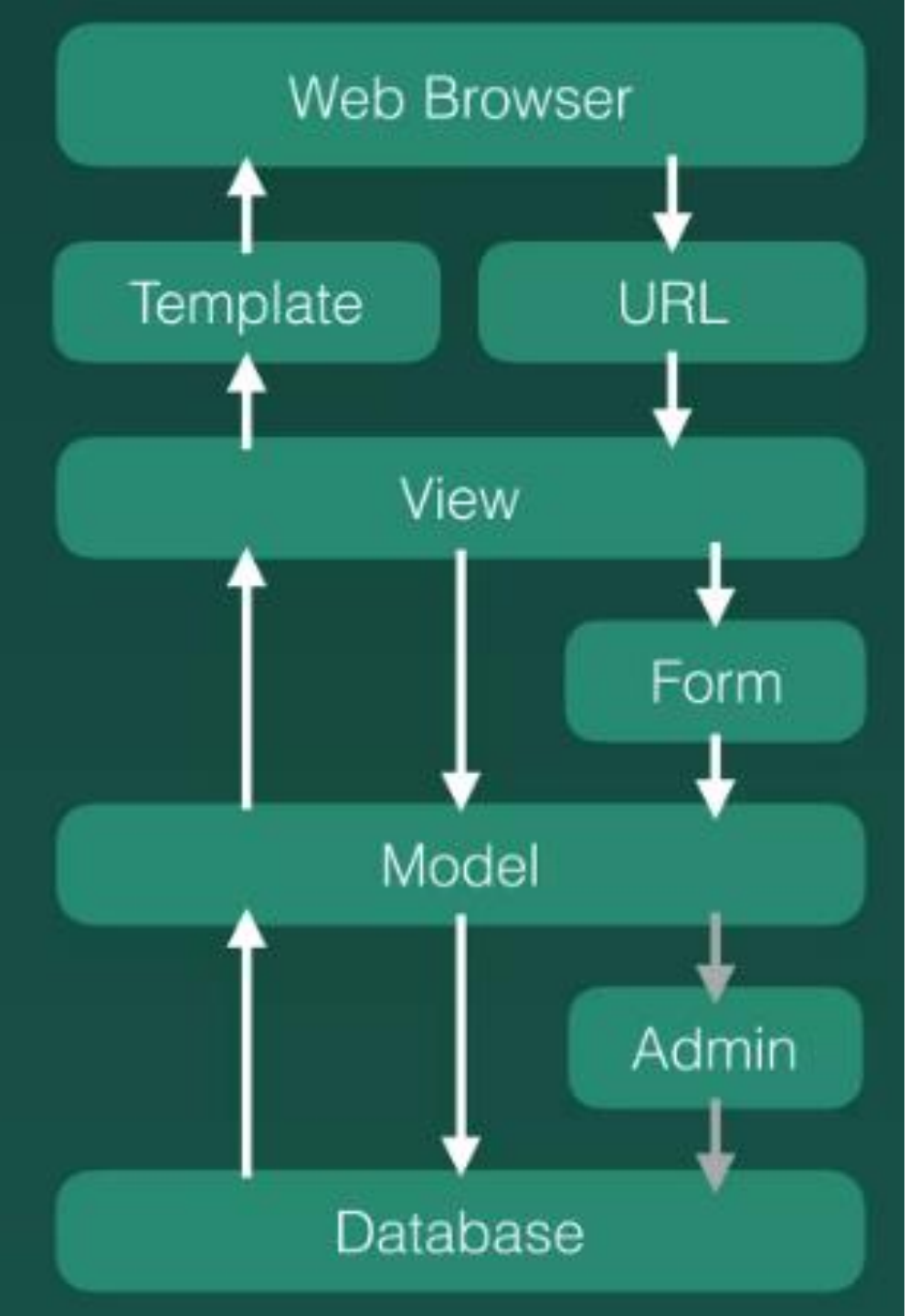
4 3 project presentations



5

Web Framework

- Django is a high-level, open-source web framework written in Python. It simplifies the development of complex, database-driven web applications.
- Django follows the Model-View-Template (MVT) architectural pattern.
- https://www.slideshare.net/slideshow/django-introduction-amp-tutorial-pdf/62644478#12



5.1 Django Architecture

1. **Models**

   Define data structure and handle database interactions.

2. **Views**

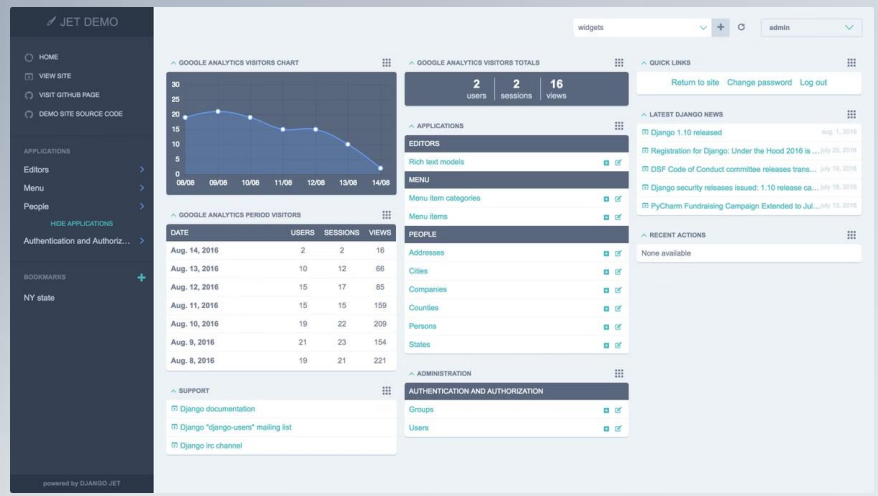
   Process requests, interact with models, and prepare data for rendering.

3. **Templates**

   Handle presentation logic and define the user interface structure.

4. **URL Configuration**

   Maps URLs to views, defining the routing mechanism.



5.2 Key Features of Django

1 ORM

Object-relational mapping simplifies database operations using Python objects.

2 URL Routing

Powerful system maps URLs to views for efficient request handling.

3 Template Engine

Separates presentation logic from business logic for dynamic HTML generation.

4 Admin Interface

Ready-to-use administration interface for managing site content.

5.3 Setting Up a Django Project

1

Install Python

Download and install Python from the official website.

2

Create Project Directory

Choose a location and create a directory for your project.

3

Set Up Virtual Environment

Create and activate a virtual environment for project isolation.

4

Install Django

Use pip to install Django within the virtual environment.



5.4 Creating a Django Project

1

Create Project

Use python django-admin startproject command to create project structure.

2

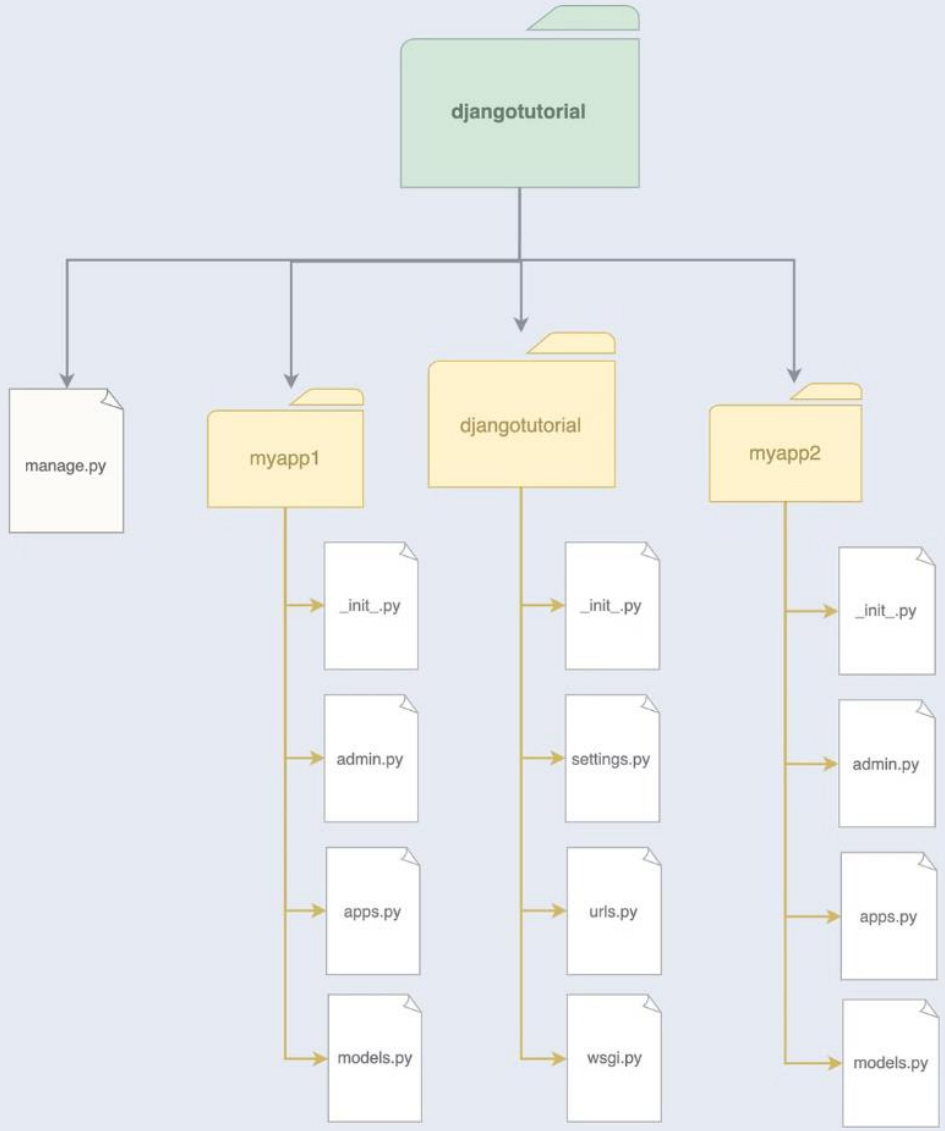
Explore Files

Examine manage.py, settings.py, urls.py, and other generated files.

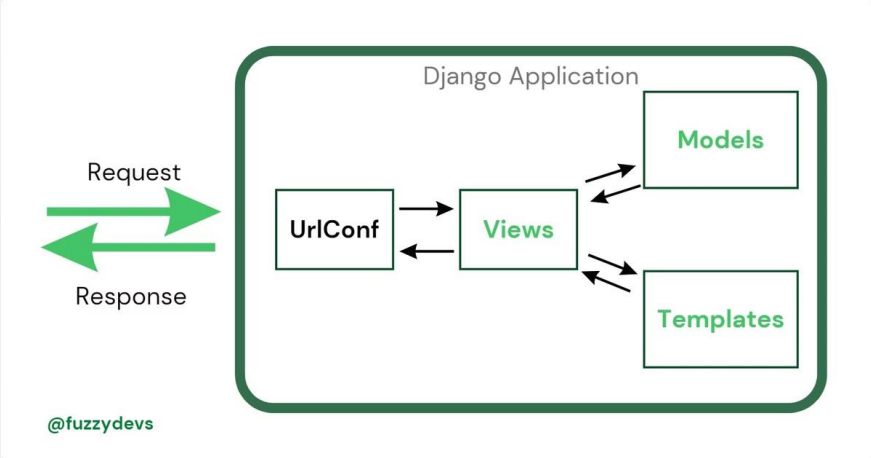
3

Run Server

Start development server with python manage.py runserver command.



5.5 Creating a Basic Django Application



1

Start App

Use **python manage.py startapp** command to create application structure.

2

Register App

Add the application name to **INSTALLED_APPS** in settings.py.

3

Define View

Create a view function in **views.py** to handle requests.

4

Configure URL

Set up URL pattern in **urls.py** to map to the view.

5.6 Django Application Structure

Core Files

- admin.py
- apps.py
- models.py
- views.py

Additional Components

- migrations directory
- tests.py

5.7 Django Forms and Authentication

Forms API

Django provides tools for handling HTML forms, data validation, and user input.

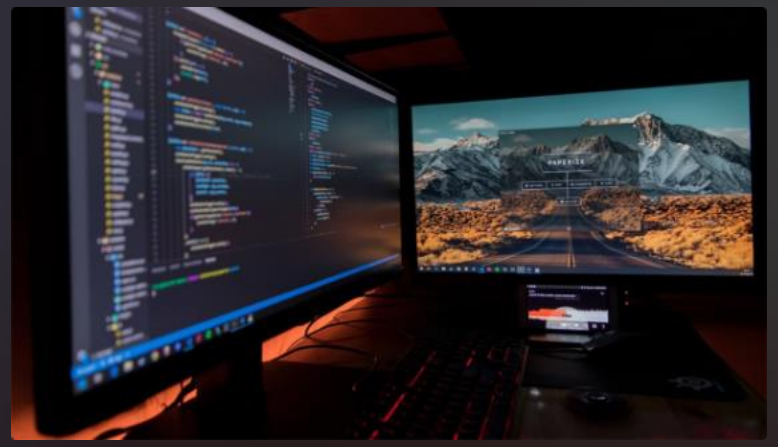
Authentication System

The built-in system manages user authentication, sessions, and permissions.

Security Features

Django includes protection against common web vulnerabilities like XSS and CSRF.





5.8 Advantage of using Django

Active Community

Large and supportive community contributes to Django's continuous improvement.

Third-Party Packages

Wide range of extensions and packages available for additional functionality.

Extensive Documentation

Comprehensive documentation and tutorials for developers of all levels.



Creating a Django Application

- We'll cover essential steps like creating a virtual environment, starting a new Django project, defining views, and configuring URLs.

Setting Up the Django Project

1. **Create Virtual Environment**
   python -m venv myenv
2. **Activate Virtual Environment**
   `source myenv/bin/activate` (Linux/Mac) or myenv\Scripts\activate (Windows)
3. **Install Django**
   pip install django
4. **List and save installed packages**
   pip freeze > requirements.txt
5. **Create Django Project**
   django-admin startproject myproject



Creating a Django Application

1

Change to Project Directory

Navigate to your project directory using **cd myproject**

2

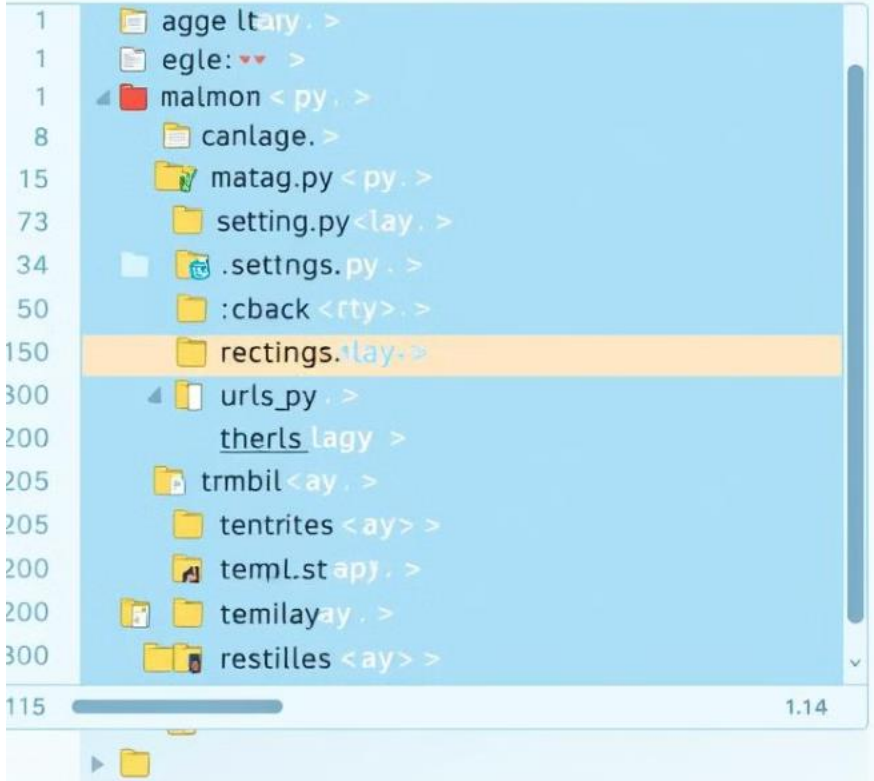
Create Django Application

Use the manage.py tool to create a new app:
python manage.py startapp myapp

3

Register the Application

Add 'myapp' to **INSTALLED_APPS** in settings.py



Defining Views and URLs

Define a View

In views.py: (In-App folder)

```
from django.http import HttpResponse

def hello(request):
        return HttpResponse("Hello, Django!“)
```

Create URL Pattern

In urls.py: (In-App folder)

```
from django.urls import path
from myapp.views import hello

urlpatterns = [
                        path('', hello, name='hello’),
                          ]
```

Defining Views and URLs

Connect the myapp URLs to the main project.
Open myproject/urls.py and include the app URLs

In urls.py: (In project)

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include(‘myapp.urls')),  # Include the myapp URLs
]
```

RUN THE SERVER

Go to Project folder
> myproject>

```
python manage.py runserver
```

Create the Template

- **Inside myapp/templates/myapp, create a file called hello.html**

  hello.html:

```
<!-- myapp/templates/myapp/hello.html -->

<!DOCTYPE html>
<html>
<head>
    <title>Hello World</title>
</head>
<body>
    <h1>Hello, World!</h1>
</body>
</html>
```

Update the view with the template rendering

- **Adjust the View**

In views.py: (In-App folder)

```
from django.http import HttpResponse
from django.shortcuts import render

def hello(request):
    #return HttpResponse("Hello, Django!“)
    return render(request, ‘myApp/hello.html’)
```



Running the Development Server

1

Navigate to Project Directory

Ensure you're in the main project directory

2

Start Server

Run: **python manage.py runserver**

3

Access Application

Visit **http://localhost:8000/hello/** in a web browser



Creating a Superuser

Create Superuser

Run:

1. **python manage.py makemigrations**
2. **python manage.py migrate**
3. **python manage.py createsuperuser**

Enter Details

Provide username, email, and password

python manage.py runserver

Access Admin Panel

Visit **http://localhost:8000/admin/** and log in

Gamma

Django Admin Panel

User Management

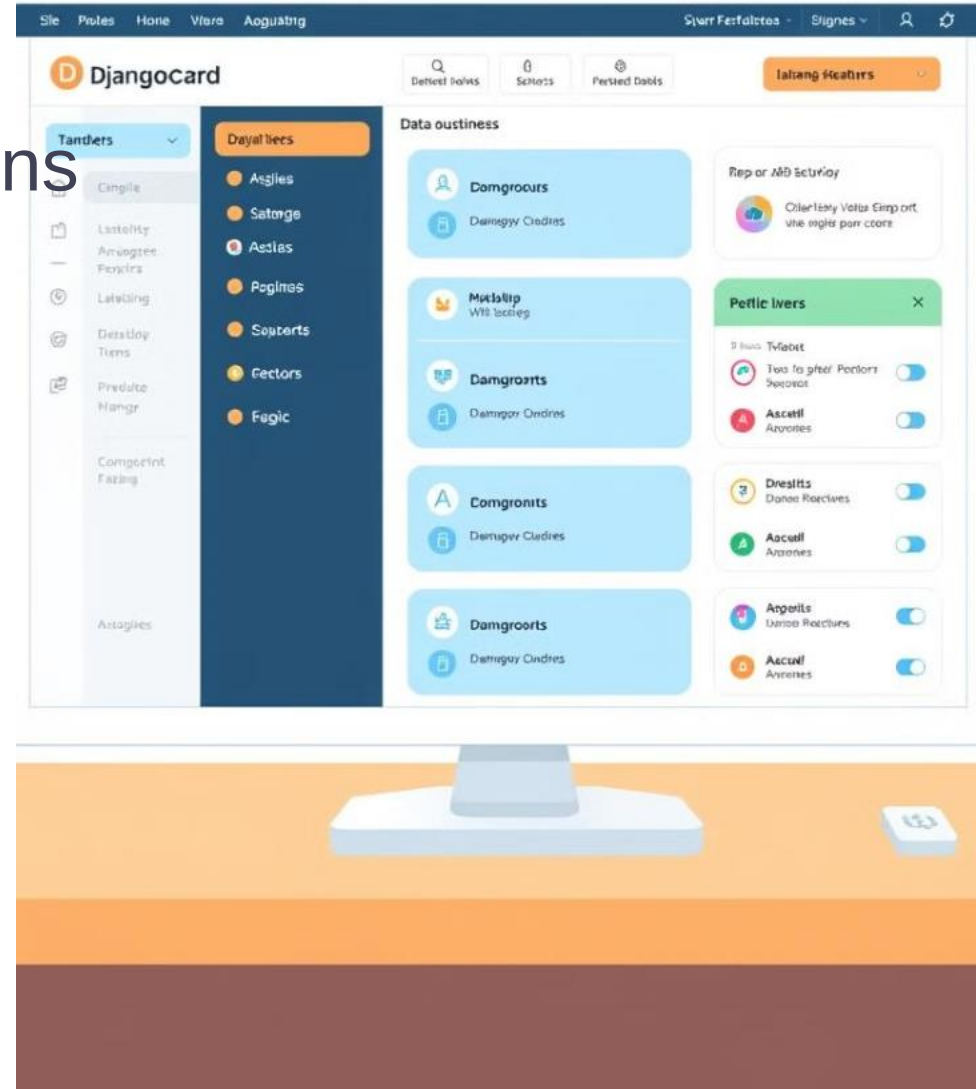
Manage user accounts and permissions

Data Management

View and edit database records

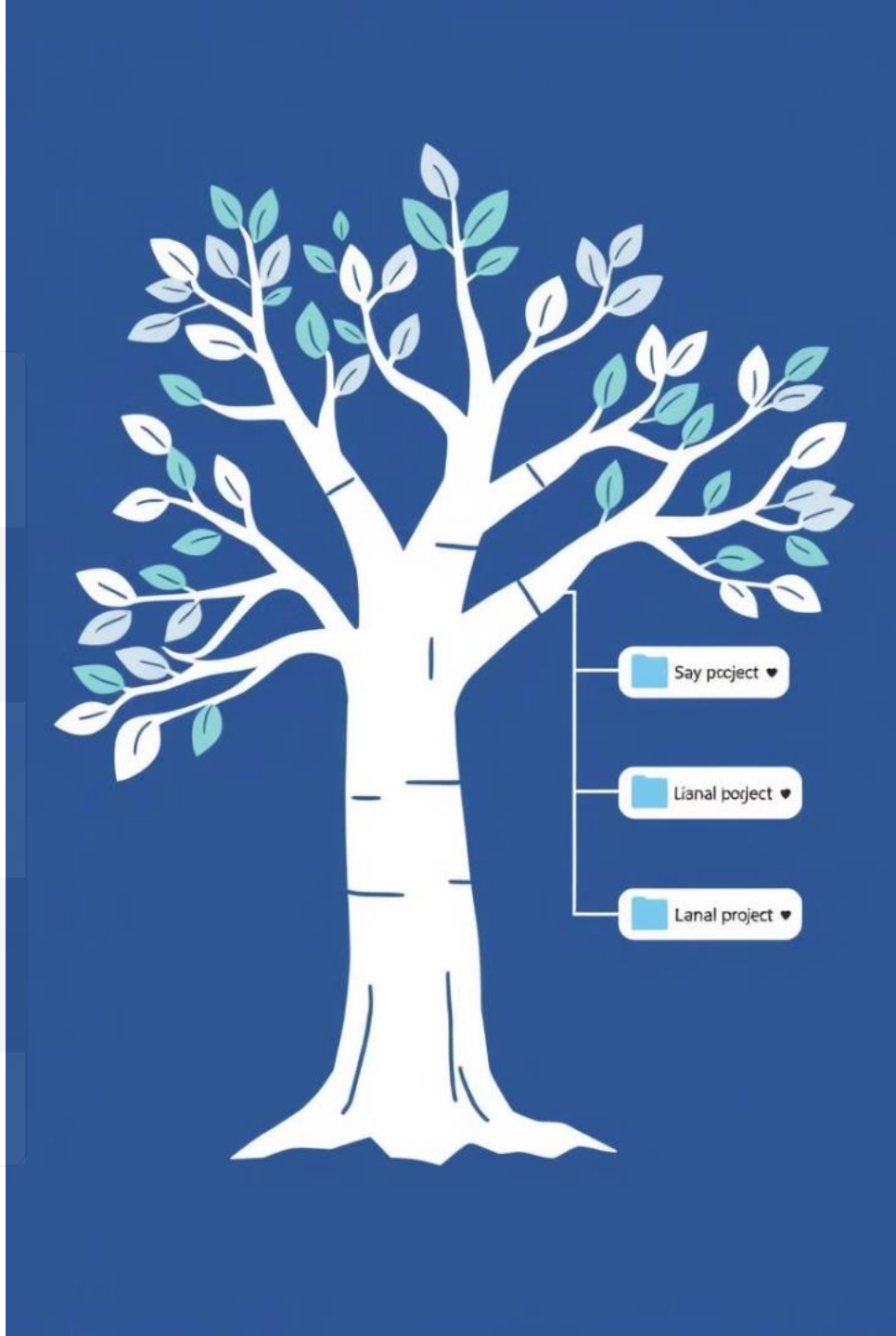
Site Configuration

Adjust site-wide settings

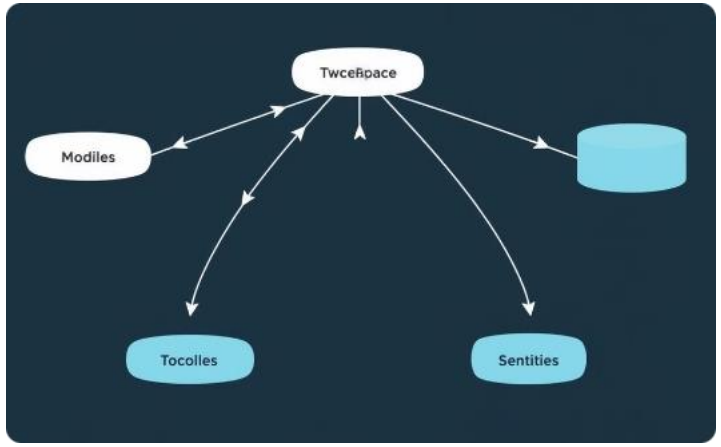


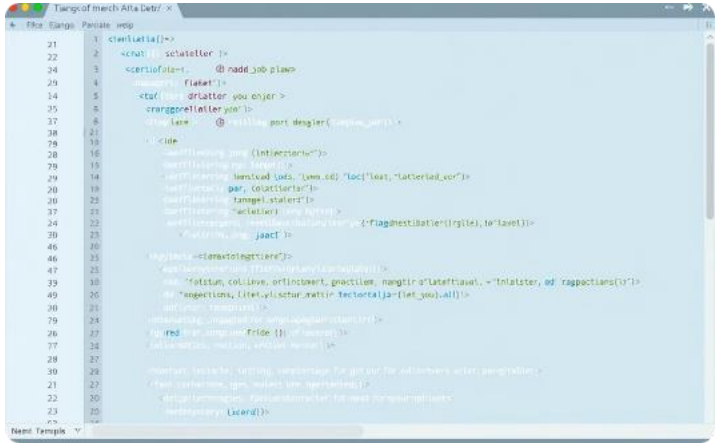
Django Project Structure

manage.py	Command-line utility for administrative tasks
myproject/	Project's Python package directory
myproject/settings.py	Project settings and configuration
myproject/urls.py	URL declarations for the project
myapp/	Application directory



Next Steps in Django Development







Database Models

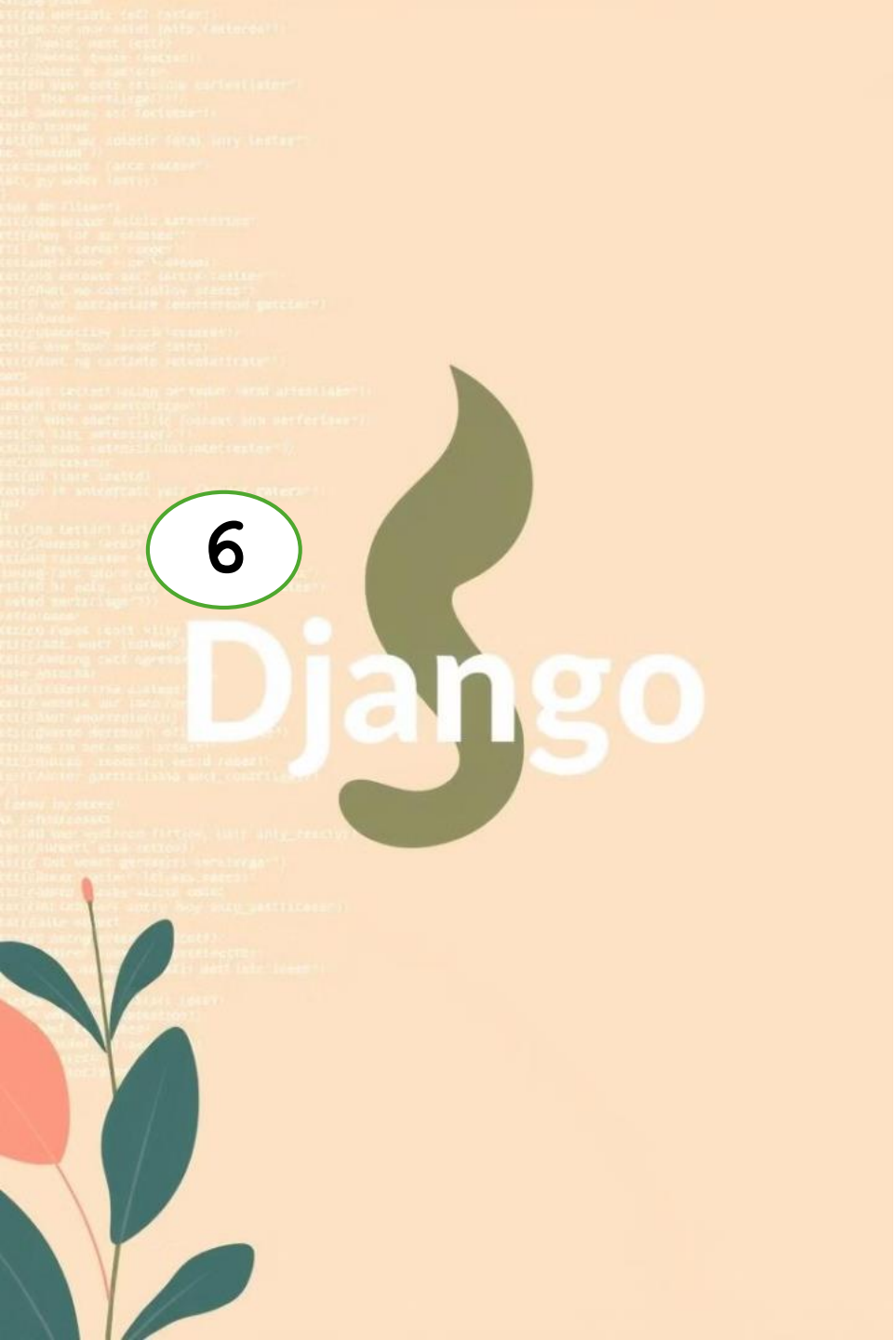
Define data models to represent your application's data structure

Templates

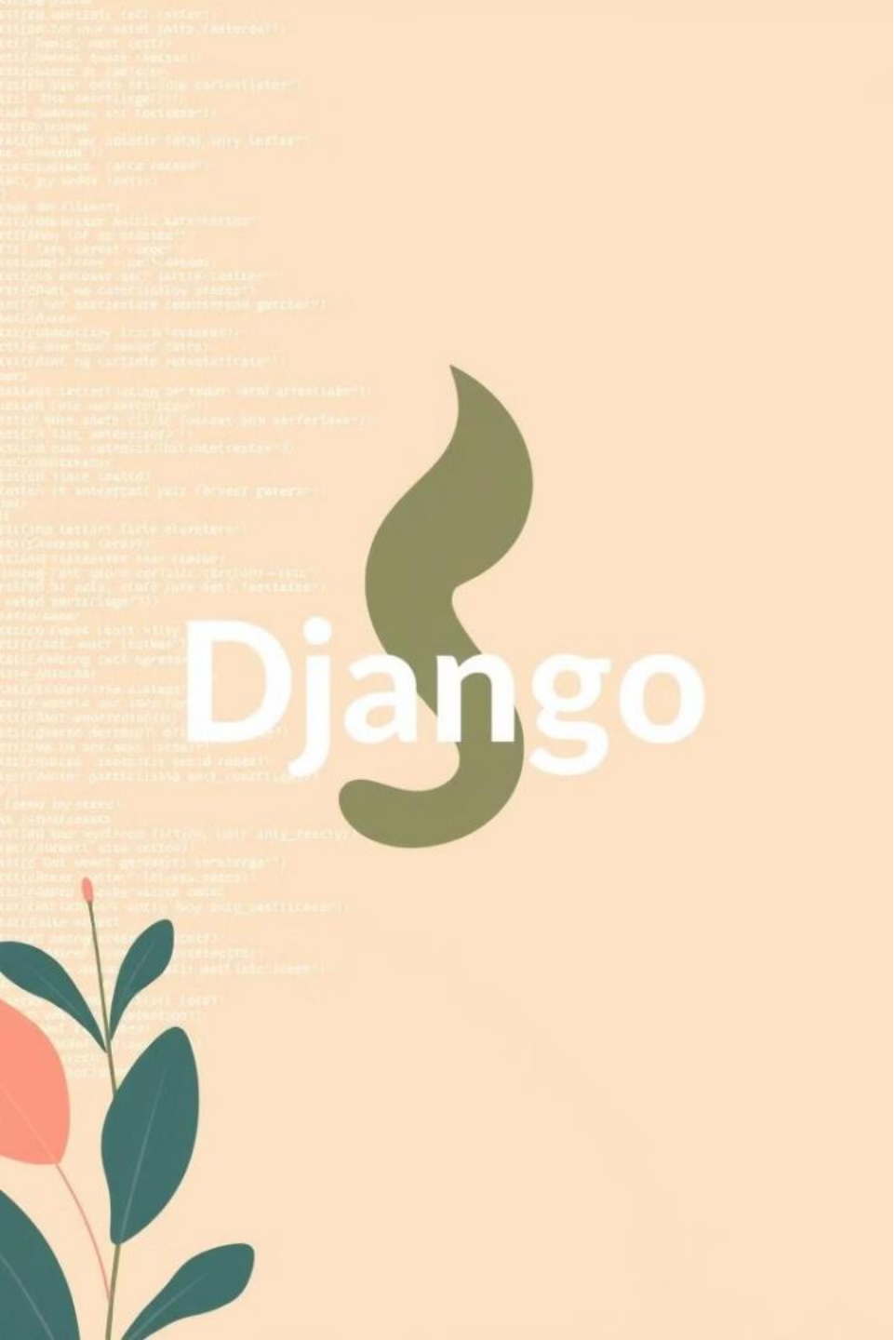
Create HTML templates to render dynamic content

Forms

Implement forms for user input and data submission



Hands-On Django Development: Models, Templates, and Forms



Hands-On Django Development: Models, Templates, and Forms

- In this detailed guide, we'll explore the practical implementation of database models, templates, and forms in Django.
- We'll focus on creating one-to-one and one-to-many relationships in our models, designing templates to render dynamic content, and implementing forms for user input.
- https://www.slideshare.net/slideshow/django-251407807/251407807
- https://www.slideshare.net/slideshow/django-arch/42611692

Model Datatypes

Django Field	Database Type (Common SQL)	Description
AutoField	INTEGER (auto-increment)	Automatically increments primary key field.
BigAutoField	BIGINT (auto-increment)	Auto-incrementing large integer field.
BigIntegerField	BIGINT	Stores large integers up to 9223372036854775807.
BinaryField	BLOB/BYTEA	Stores raw binary data.
BooleanField	BOOLEAN	Stores True/False values.
CharField	VARCHAR	Stores a string with a max length.
DateField	DATE	Stores date without time.

Model Datatypes

DateTimeField	TIMESTAMP/DATETIME	Stores date and time.
DecimalField	DECIMAL	Fixed-precision decimal numbers for exact arithmetic.
DurationField	INTERVAL	Stores time durations.
EmailField	VARCHAR	Specialized string field for email addresses.
FileField	VARCHAR	Path to an uploaded file.
FilePathField	VARCHAR	Path to a local file system file.
FloatField	FLOAT	Stores floating-point numbers.
ImageField	VARCHAR	Path to an uploaded image file.
IntegerField	INTEGER	Stores small to medium-sized integers.
JSONField	JSON (PostgreSQL, MySQL)	Stores JSON data (requires compatible database).

Model Datatypes

SmallIntegerField	SMALLINT	Stores small integer values.
TextField	TEXT	Stores large text data.
TimeField	TIME	Stores a time without date.
URLField	VARCHAR	Specialized string field for URLs.
UUIDField	CHAR(32)/UUID	Stores universally unique identifiers (UUID).
ForeignKey	INTEGER/UUID (with relation)	Foreign key reference to another model.
OneToOneField	INTEGER/UUID (with relation)	One-to-one reference to another model.
ManyToManyField	Multiple Foreign Keys	Represents a many-to-many relationship between models.

Database Models: One-to-One Relationships

- One-to-one relationship example. Imagine we're building a user profile system where each user has one profile.

1

Step 1: Define User Model

```
from django.contrib.auth.models import User
from django.db import models
class Profile(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE)
    bio = models.TextField(max_length=500, blank=True)
    location = models.CharField(max_length=30, blank=True)
    birth_date = models.DateField(null=True, blank=True)
```

2

Step 2: Create and Run Migrations

```
python manage.py makemigrations
python manage.py migrate
```

3

Step 3: Use the Model

```
    user = User.objects.create_user('username', 'email@example.com', 'password’)
    Profile.objects.create(user=user, bio='Django developer', location='New York')
```

Database Models: One-to-One Relationships

1

Step 1: Define User Model

```
from django.contrib.auth.models import User
from django.db import models
class Profile(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE)
    bio = models.TextField(max_length=500, blank=True)
    location = models.CharField(max_length=30, blank=True)
    birth_date = models.DateField(null=True, blank=True)
```

Database Models: One-to-One Relationships

2

Step 2: Create and Run Migrations

```
python manage.py makemigrations
python manage.py migrate
```

Database Models: One-to-One Relationships

2

Step 3: Use the Model

```
user = User.objects.create_user('username', 'email@example.com', 'password’)

Profile.objects.create(user=user, bio='Django developer', location='New York')
```

Database Models: One-to-One Relationships

- One-to-one relationship example. Imagine we're building a user profile system where each user has one profile.

1

Step 1: Define User Model

```
from django.contrib.auth.models import User
from django.db import models
class Profile(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE)
    bio = models.TextField(max_length=500, blank=True)
    location = models.CharField(max_length=30, blank=True)
    birth_date = models.DateField(null=True, blank=True)
```

2

Step 2: Create and Run Migrations

```
python manage.py makemigrations
python manage.py migrate
```

3

Step 3: Use the Model

```
    user = User.objects.create_user('username', 'email@example.com', 'password’)
    Profile.objects.create(user=user, bio='Django developer', location='New York')
```

Register the model on the Admin site

> Admin.py file

```
1    from django.contrib import admin
2    from .models import Profile
3
4    # Register your models here.
5    admin.site.register(Profile)
6
```

Update the PROJECT SET UP Accordingly



Made with Gamma

View (Update views.py inside the App)

```python
from django.shortcuts import render, get_object_or_404, redirect
from django.contrib.auth.models import User
from .models import Profile
from .forms import ProfileForm

# Create your views here.
from django.http import HttpResponse

def hello(request):
    #return HttpResponse("Hello, Django!")
    return render(request, 'myseedApp/hello.html')



# --------------------------
# List of users
# --------------------------

def user_list(request):
    users = User.objects.all().select_related()
    return render(request, "myseedApp/user_list.html", {"users": users})
```

View (Update views.py inside the App)

```python
# ----------------------------
# Create Profile
# ----------------------------
def profile_create(request, user_id):
    user = get_object_or_404(User, id=user_id)

    # Prevent duplicate profile creation
    if hasattr(user, "profile"):
        return redirect("myseedApp/profile_detail", pk=user.profile.pk)

    if request.method == "POST":
        form = ProfileForm(request.POST)
        if form.is_valid():
            profile = form.save(commit=False)
            profile.user = user
            profile.save()
            return redirect("myseedApp/profile_detail", pk=profile.pk)
    else:
        form = ProfileForm()

    return render(request, "myseedApp/profile_form.html", {"form": form, "action": "Create"})
```

View (Update views.py inside the App)

```
# ---------------------------
# Read / Fetch Profile
# ---------------------------
def profile_detail(request, pk):
    profile = get_object_or_404(Profile, pk=pk)
    return render(request, "myseedApp/profile_detail.html", {"profile": profile})
```

View (Update views.py inside the App)

```python
# --------------------------
# Update Profile
# --------------------------
def profile_update(request, pk):
    profile = get_object_or_404(Profile, pk=pk)

    if request.method == "POST":
        form = ProfileForm(request.POST, instance=profile)
        if form.is_valid():
            form.save()
            return redirect("myseedApp/profile_detail", pk=profile.pk)
    else:
        form = ProfileForm(instance=profile)

    return render(request, "myseedApp/profile_form.html", {"form": form, "action": "Update"})
```

View (Update views.py inside the App)

```python
# ----------------------------
# Delete Profile
# ----------------------------
def profile_delete(request, pk):
    profile = get_object_or_404(Profile, pk=pk)

    if request.method == "POST":
        profile.delete()
        return redirect("home")  # Change as needed

    return render(request, "myseedApp/profile_delete.html", {"profile": profile})
```

Form (Create forms.py inside of the App)

```python
from django import forms
from .models import Profile

class ProfileForm(forms.ModelForm):
    class Meta:
        model = Profile
        fields = ['bio', 'location', 'birth_date']
```

urls (Update urls.py inside of the App)

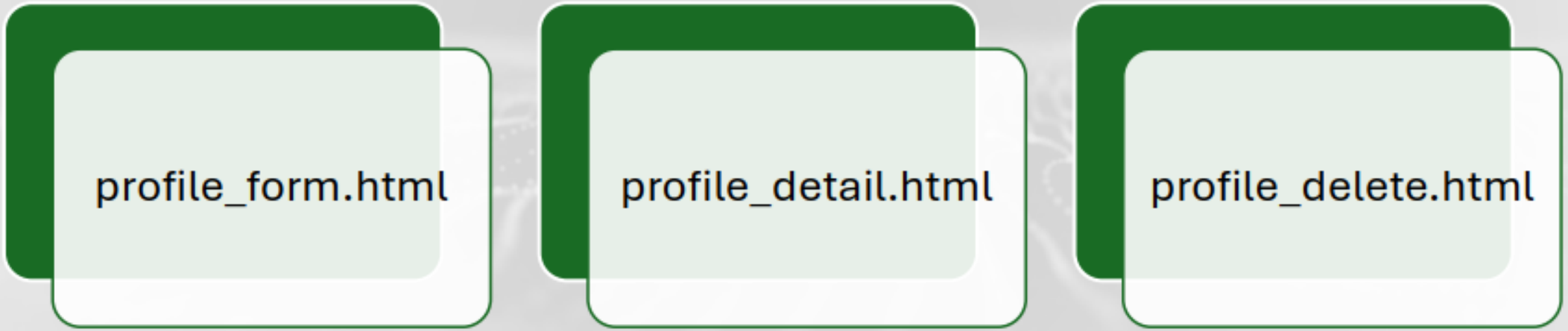
```python
from django.urls import path
from myseedApp.views import hello
from . import views

urlpatterns = [
    path('hello/', hello, name='hello'),

    path('users/', views.user_list, name='user_list'),

    path("profile/create/<int:user_id>/", views.profile_create, name="profile_create"),
    path("profile/<int:pk>/", views.profile_detail, name="profile_detail"),
    path("profile/<int:pk>/update/", views.profile_update, name="profile_update"),
    path("profile/<int:pk>/delete/", views.profile_delete, name="profile_delete"),
]
```

Templates (Add files inside of the templates)



User_list.html

```
<h2>All Users</h2>

<table border="1" cellpadding="8">
    <tr>
        <th>Username</th>
        <th>Email</th>
        <th>Profile</th>
        <th>Action</th>
    </tr>

    {% for user in users %}
    <tr>
        <td>{{ user.username }}</td>
        <td>{{ user.email }}</td>

        <td>
            {% if user.profile %}
                ✓ Profile Exists
            {% else %}
                ✕ No Profile
            {% endif %}
        </td>
        <td>
            {% if user.profile %}
                <a href="{% url 'profile_detail' user.profile.pk %}">View</a> |
                <a href="{% url 'profile_update' user.profile.pk %}">Edit</a>
            {% else %}
                <a href="{% url 'profile_create' user.id %}">Create Profile</a>
            {% endif %}
        </td>
    </tr>
    {% endfor %}
</table>
```

Profile_form.html

```
1   <h2>{{ action }} Profile</h2>
2
3   <form method="POST">
4       {% csrf_token %}
5       {{ form.as_p }}
6       <button type="submit">{{ action }}</button>
7   </form>
8
```

Profile_detail.html

```
<h2>Profile Details</h2>

<p><strong>User:</strong> {{ profile.user.username }}</p>
<p><strong>Bio:</strong> {{ profile.bio }}</p>
<p><strong>Location:</strong> {{ profile.location }}</p>
<p><strong>Birth date:</strong> {{ profile.birth_date }}</p>

<a href="{% url 'profile_update' profile.pk %}">Edit</a> |
<a href="{% url 'profile_delete' profile.pk %}">Delete</a>
```

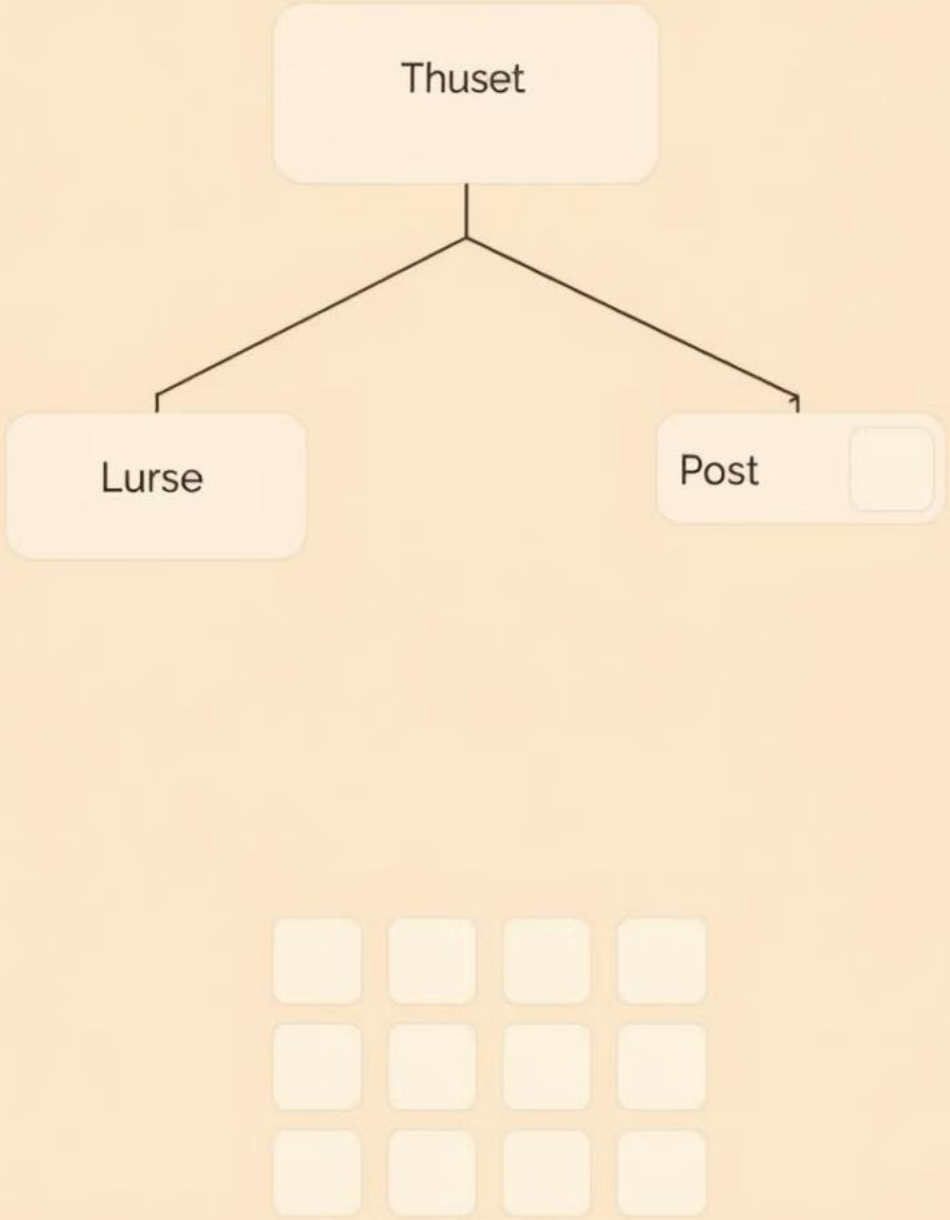
Profile_delete.html

```
<h2>Delete Profile</h2>

<p>Are you sure you want to delete {{ profile.user.username }}'s profile?</p>

<form method="POST">
    {% csrf_token %}
    <button type="submit">Confirm Delete</button>
</form>
```

Made with Gamma



Database Models: One-to-Many Relationships

- Now, let's implement a one-to-many relationship. We'll create a blog post system where each user can have multiple posts:

1. **Step 1: Define Post Model**

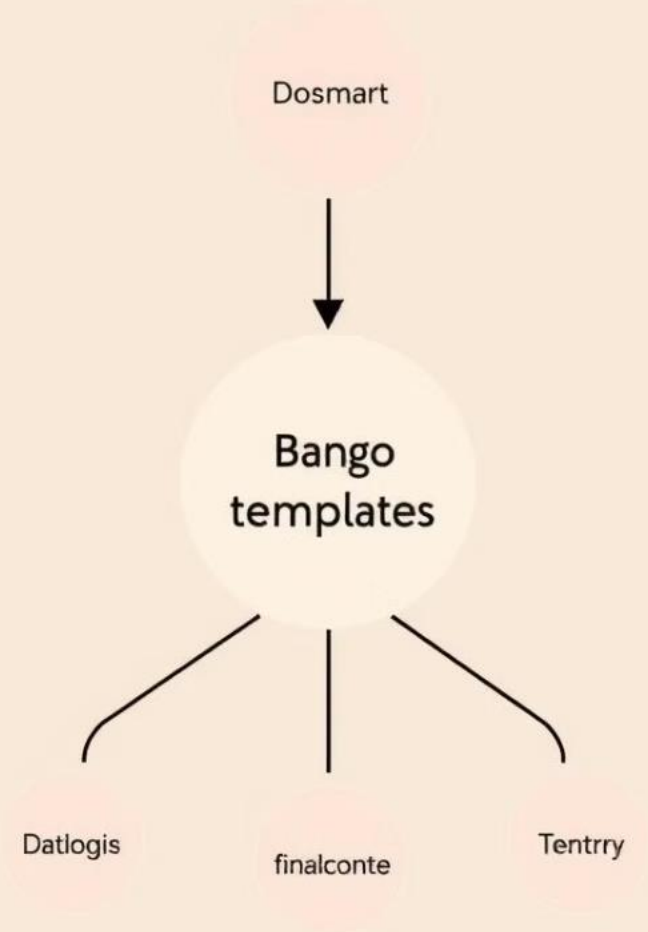
```
from django.contrib.auth.models import User
from django.db import models
class Post(models.Model):
        author = models.ForeignKey(User, on_delete=models.CASCADE)
        title = models.CharField(max_length=200)
        content = models.TextField()
        created_at = models.DateTimeField(auto_now_add=True)
```

2. **Step 2: Create and Run Migrations**

```
python manage.py makemigrations
python manage.py migrate
```

3. **Step 3: Use the Model**

```
user = User.objects.get(username='username')
Post.objects.create(author=user, title='My First Post',
content='Hello, Django!')
```



Templates: Creating a Base Template

- Templates in Django allow us to render dynamic content. Let's create a base template that other templates can extend:

Step 1: Create base.html

Create a file named base.html in your template's directory:

Step 2: Add HTML Structure

My Django Site

{% block content %} {% endblock %}

© 2023 My Django Site



Templates: Extending the Base Template

- Now that we have a base template,
  let's create a template for our blog posts that extends it:

Step 1: Create post_list.html

Create a file named post_list.html in your templates directory:

Step 2: Extend Base Template

{% extends 'base.html' %} {% block title %}Blog Posts{% endblock %}
{% block content %}

Blog Posts

{% for post in posts %}

{{ post.title }}

By {{ post.author.username }} on {{ post.created_at }}

{{ post.content|truncatewords:30 }}

{% endfor %}

{% endblock %}

More examples with Forms and Extending Template

Forms: Creating a ModelForm

- Django's forms make it easy to handle user input. Let's create a form for our Post model:

1

Step 1: Create forms.py

Create a file named forms.py in your app directory:

2

Step 2: Define PostForm

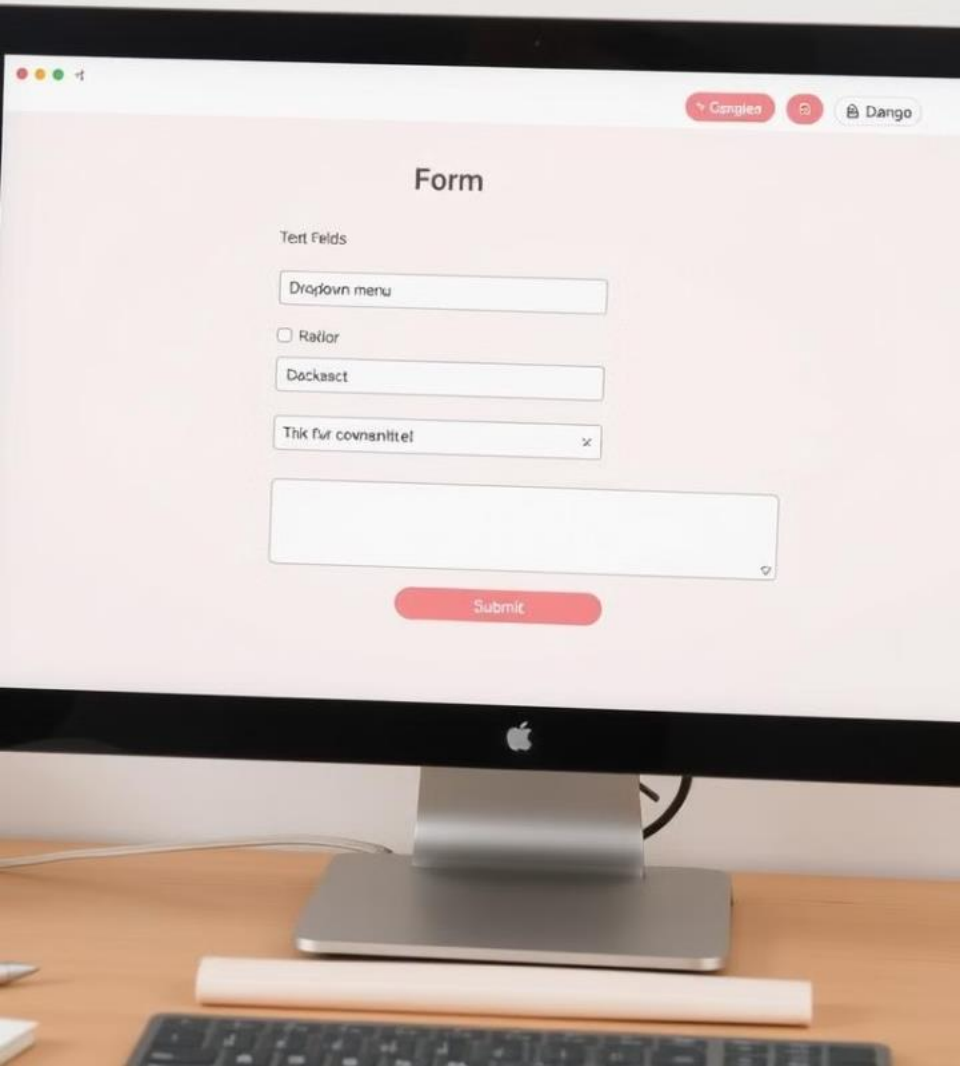
```
from django import forms
from .models import Post
class PostForm(forms.ModelForm):
        class Meta:
            model = Post
             fields = ['title', 'content']
```

3

Step 3: Use the Form in a View

```python
from django.shortcuts import render, redirect
from .forms import PostForm

def create_post(request):
    if request.method == 'POST':
        form = PostForm(request.POST)
        if form.is_valid():
            post = form.save(commit=False)
            post.author = request.user
            post.save()
            return redirect('post_list')
    else:
        form = PostForm()
    return render(request, 'create_post.html', {'form': form})
```



Forms: Creating a Template for the Form

- Now that we have our PostForm, let's create a template to display it:

Step 1: Create create_post.html

Create a file named create_post.html in your templates directory:

Step 2: Add Form to Template

{% extends 'base.html' %} {% block title %}Create Post{% endblock %} {% block content %}

Create a New Post

{% csrf_token %} {{ form.as_p }}

Create Po...

{% endblock %}

URL Configuration

- To make our views accessible, we need to configure URLs. Here's how to set up URLs for our post list and create post views:

1

Step 1: Open urls.py

Open the urls.py file in your app directory (or create one if it doesn't exist):

2

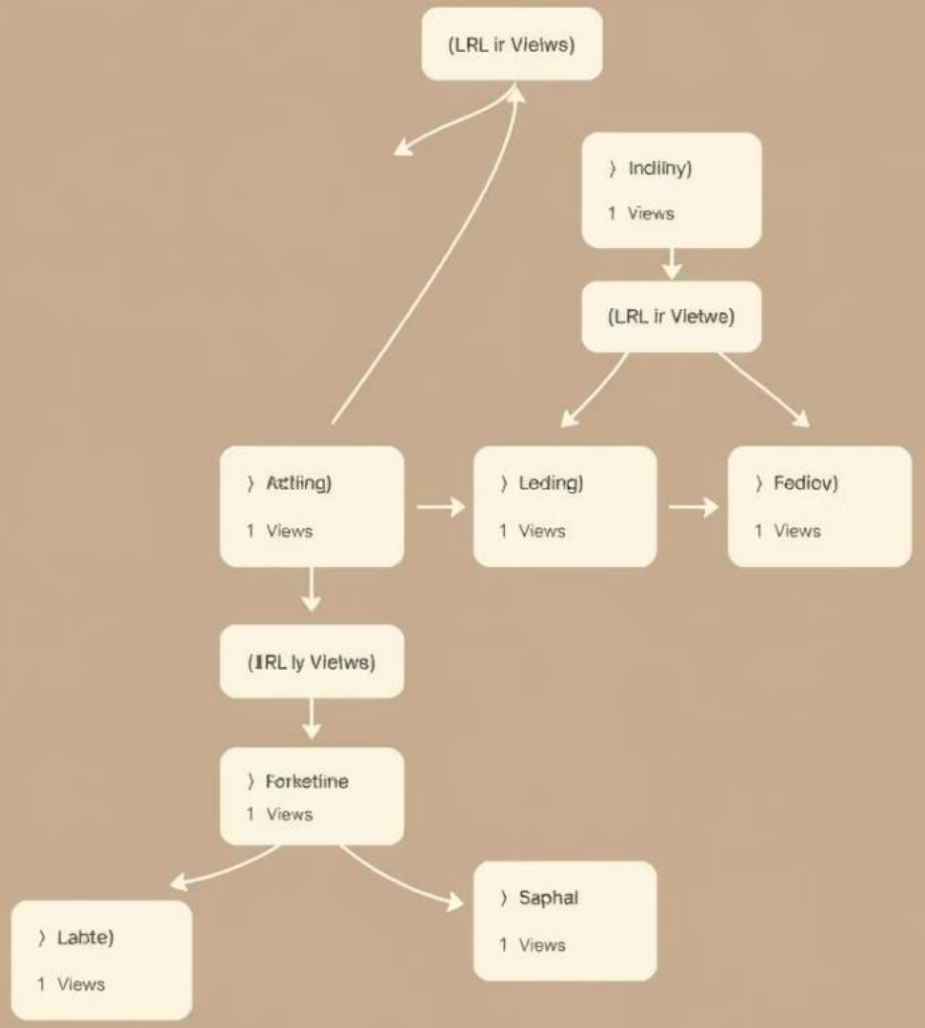
Step 2: Import Views

from django.urls import path from . import views

3

Step 3: Define URL Patterns

```
urlpatterns = [
        path('posts/', views.post_list, name='post_list’),
        path('posts/create/', views.create_post, name='create_post’),
]
```



Launching the App

- Finally, let's launch our Django app:

1

Step 1: Run the Development Server

python manage.py runserver

2

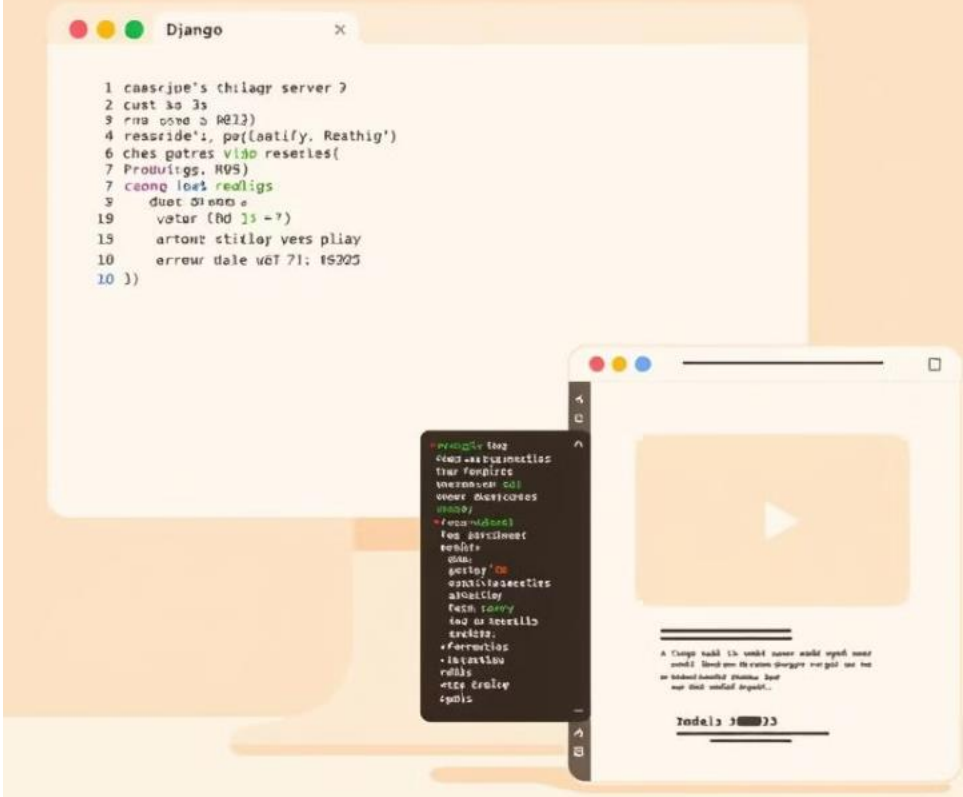
Step 2: Access the App

Open a web browser and go to http://127.0.0.1:8000/posts/ to view the post list

3

Step 3: Create a New Post

Navigate to http://127.0.0.1:8000/posts/create/ to create a new post



Database Set-up

- To configure your Django project to use **MySQL** or **PostgreSQL** instead of SQLite, follow these steps:
- Install the required Database Adapter
- **For MySQL**
  - pip install mysqlclient or pip install PyMySQL

❑ **Update your settings.py File in your Project.**

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql',
        'NAME': 'your_database_name',  # Replace with your database name
        'USER': 'your_username',       # Replace with your database username
        'PASSWORD': 'your_password',   # Replace with your database password
        'HOST': '127.0.0.1',           # Or use 'localhost'
        'PORT': '3306',                # Default MySQL port
        'OPTIONS': {
            'init_command': "SET sql_mode='STRICT_TRANS_TABLES'",
        },
    }
}
```

Database Set-up

- To configure your Django project to use **MySQL** or **PostgreSQL** instead of SQLite, follow these steps:
- Install the required Database Adapter
- **For PostgreSQL**
  - pip install psycopg2 or pip install psycopg2-binary

❑ **Update your settings.py File in your Project.**

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'your_database_name',  # Replace with your database name
        'USER': 'your_username',       # Replace with your database username
        'PASSWORD': 'your_password',   # Replace with your database password
        'HOST': '127.0.0.1',           # Or use 'localhost'
        'PORT': '5432',                # Default PostgreSQL port
    }
}
```

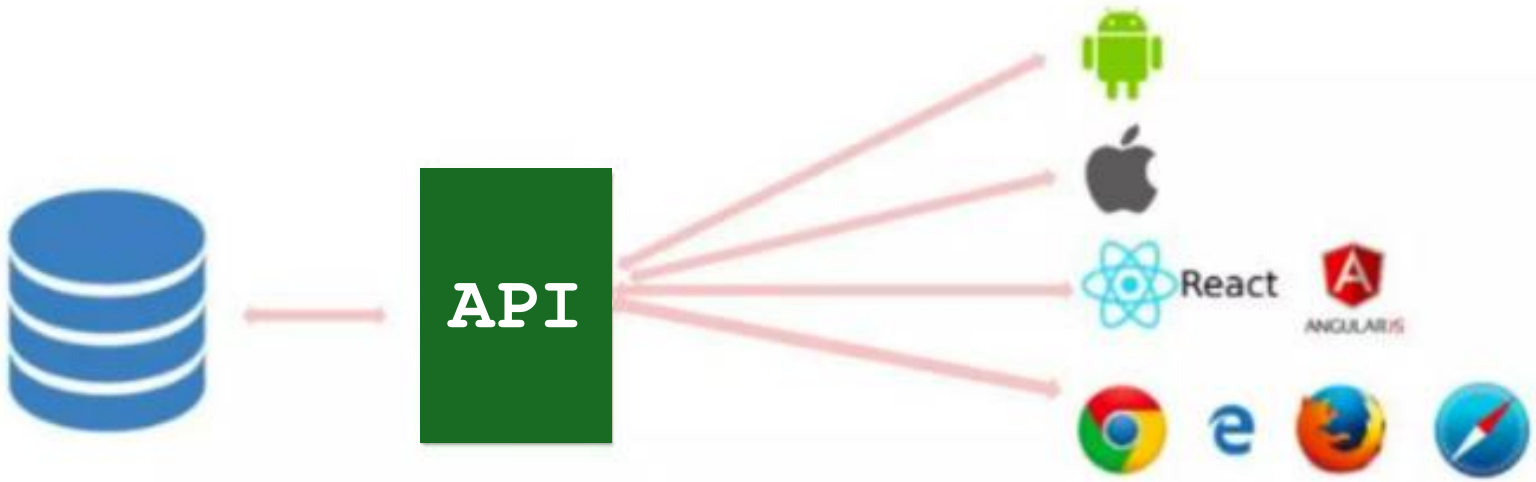
Project Mapping

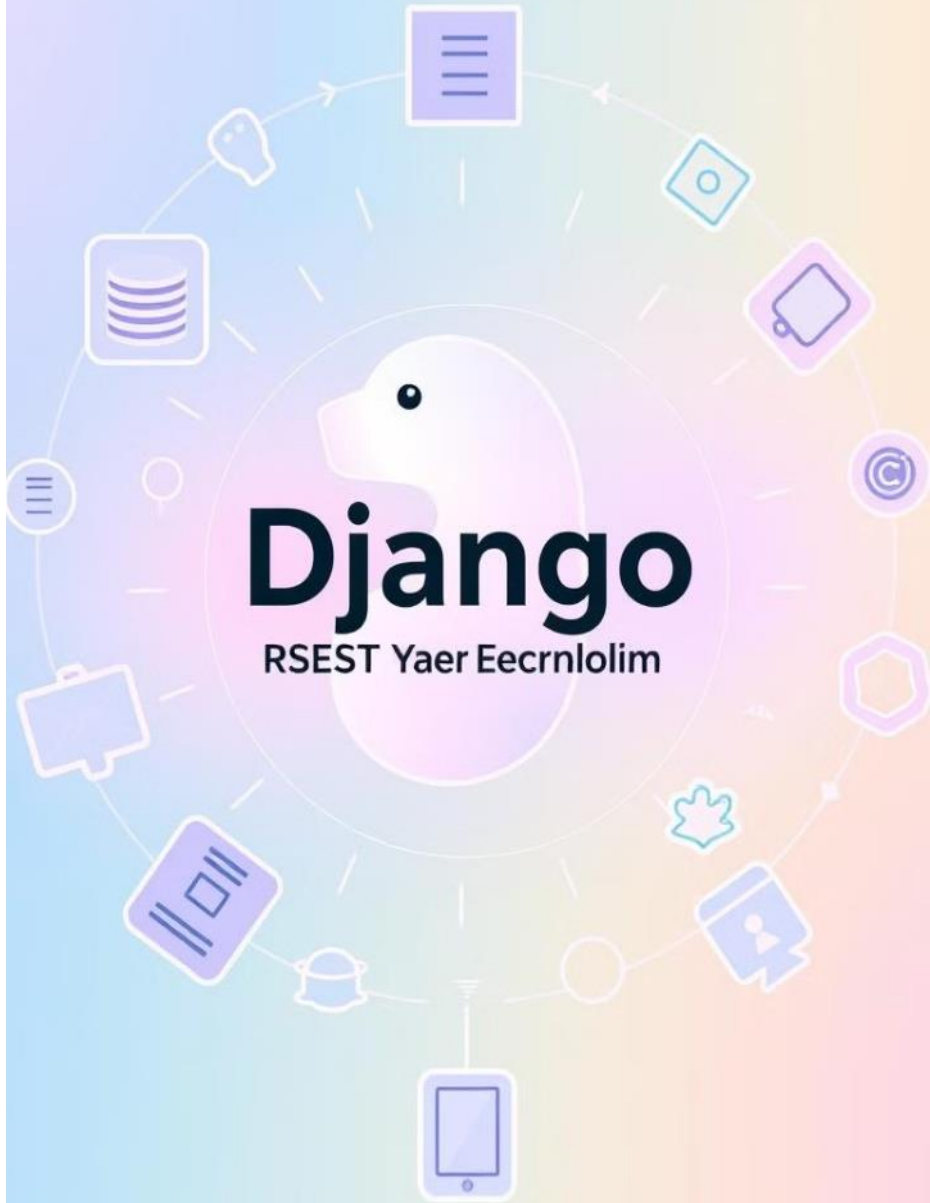
APPLICATION PROGRAMMING INTERFACE (API)

7

API

- It specifies how software components should interact.
- It is a set of routines, protocols, and tools for building software applications.





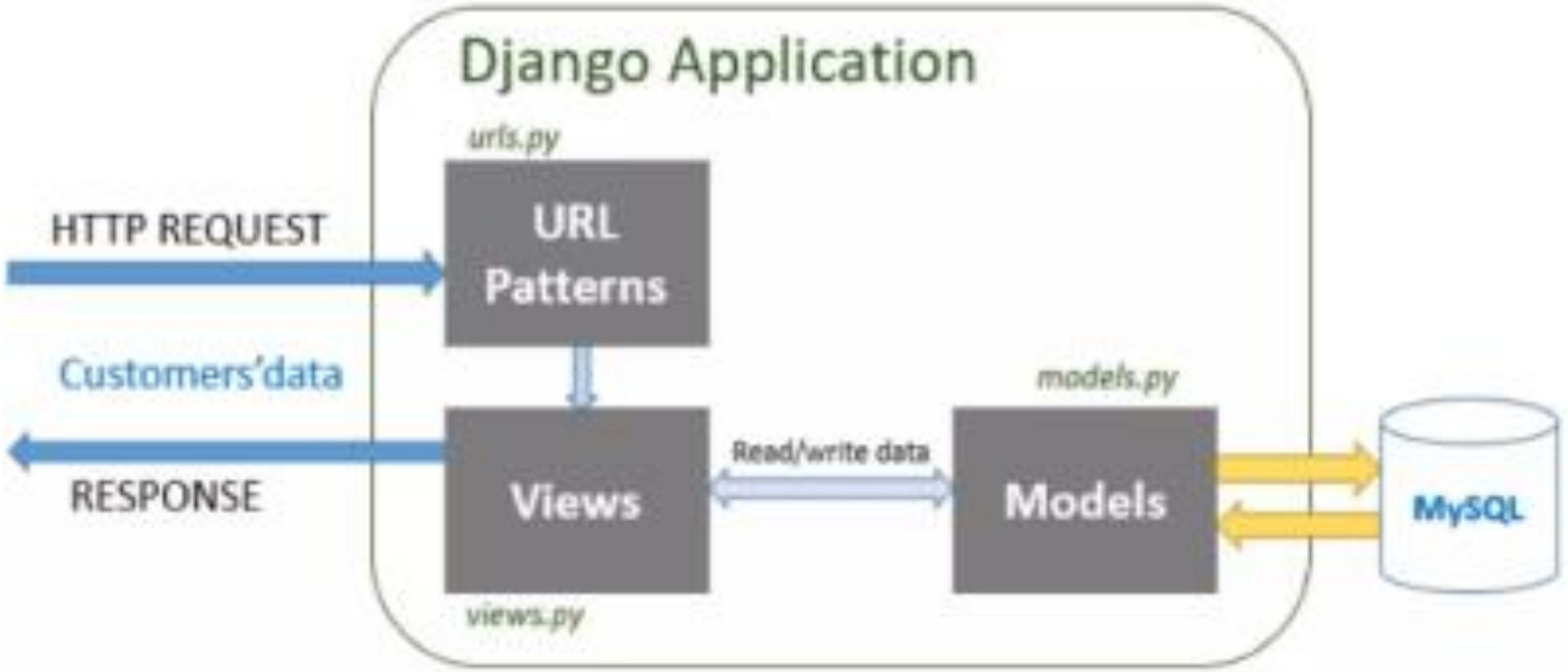
RESTful APIs with Django

- Django REST Framework (DRF) is a powerful **toolkit** for creating Web APIs.
- It simplifies API development by providing **reusable components** and utilities for tasks like serialization, authentication, routing, and request/response handling.
- DRF extends Django's functionality to make building robust and scalable RESTful APIs easier.





API Design Best Practices



Serializers

- Serializers allow complex data such as query sets and model instances to be converted to native Python datatypes JSON, XML and etc.

  - ✓ **ModelSerializers** (maps model fields to JSON, standard CRUD representation)
  - ✓ HyperlinkedModelSerializer (like above, but uses URLs for relations instead of PKs)
  - ✓ ListSerializer (handles a list of objects at once, for bulk operations)

Viewset

- Viewset combines the logic for a set of related views in a single class. (Generic or **Viewsets** – group multiple actions in one class)
  - ✓ List
  - ✓ Create
  - ✓ Retrieve
  - ✓ Update
  - ✓ Partial Update
  - ✓ Destroy



Routers

- To avoid creating endless “list”, “details” and “edit” URLs, the DRF routers bundle all the URLs needed for a given viewset into one line per viewset.

  - ✓ SimpleRouter (generates URLs for ViewSets without an extra API root view)
  - ✓ **DefaultRouter** (like above, plus a default API root view listing all registered routes)

Security

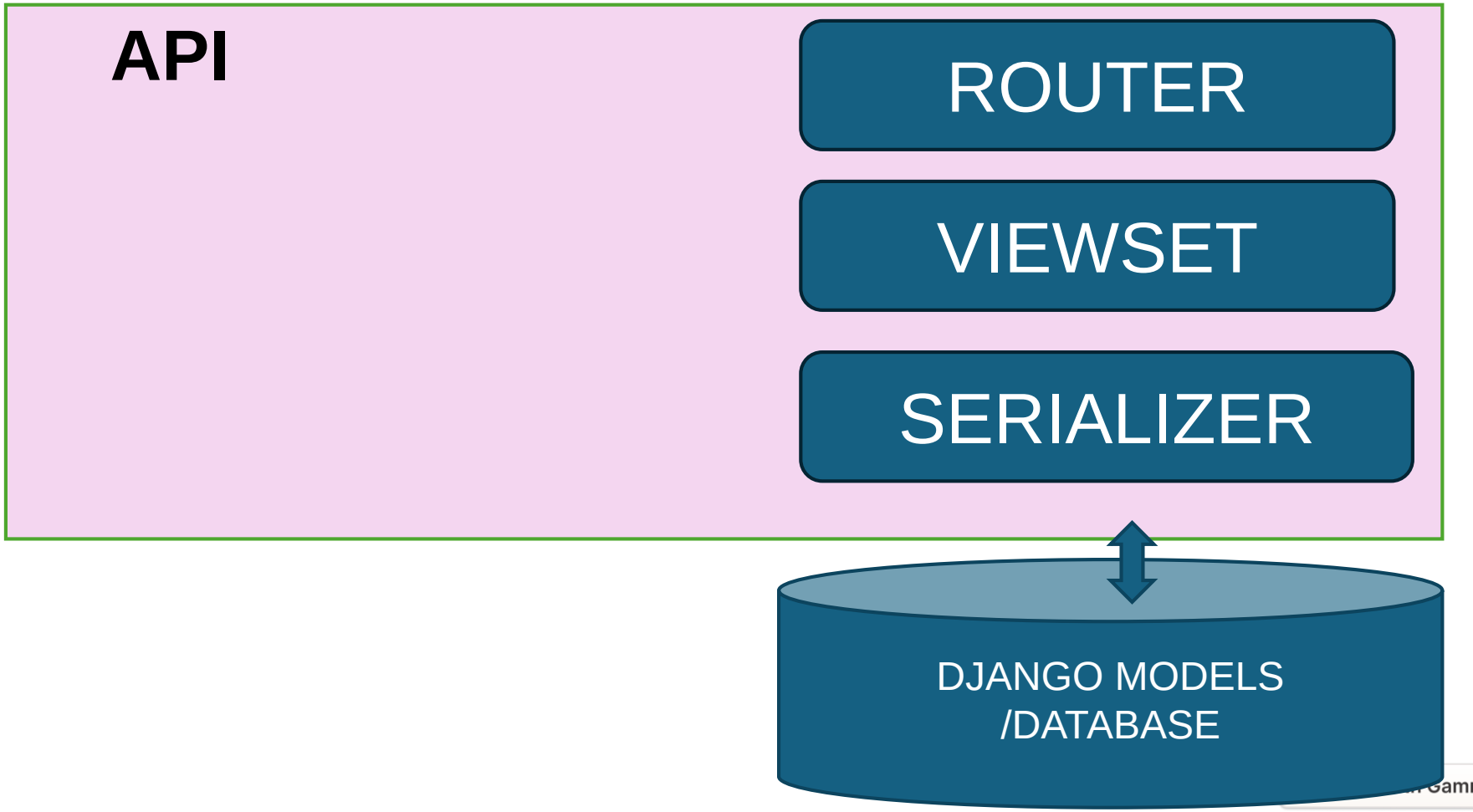
- Security is managed by Authentication and Permissions. Authentication schemes are:

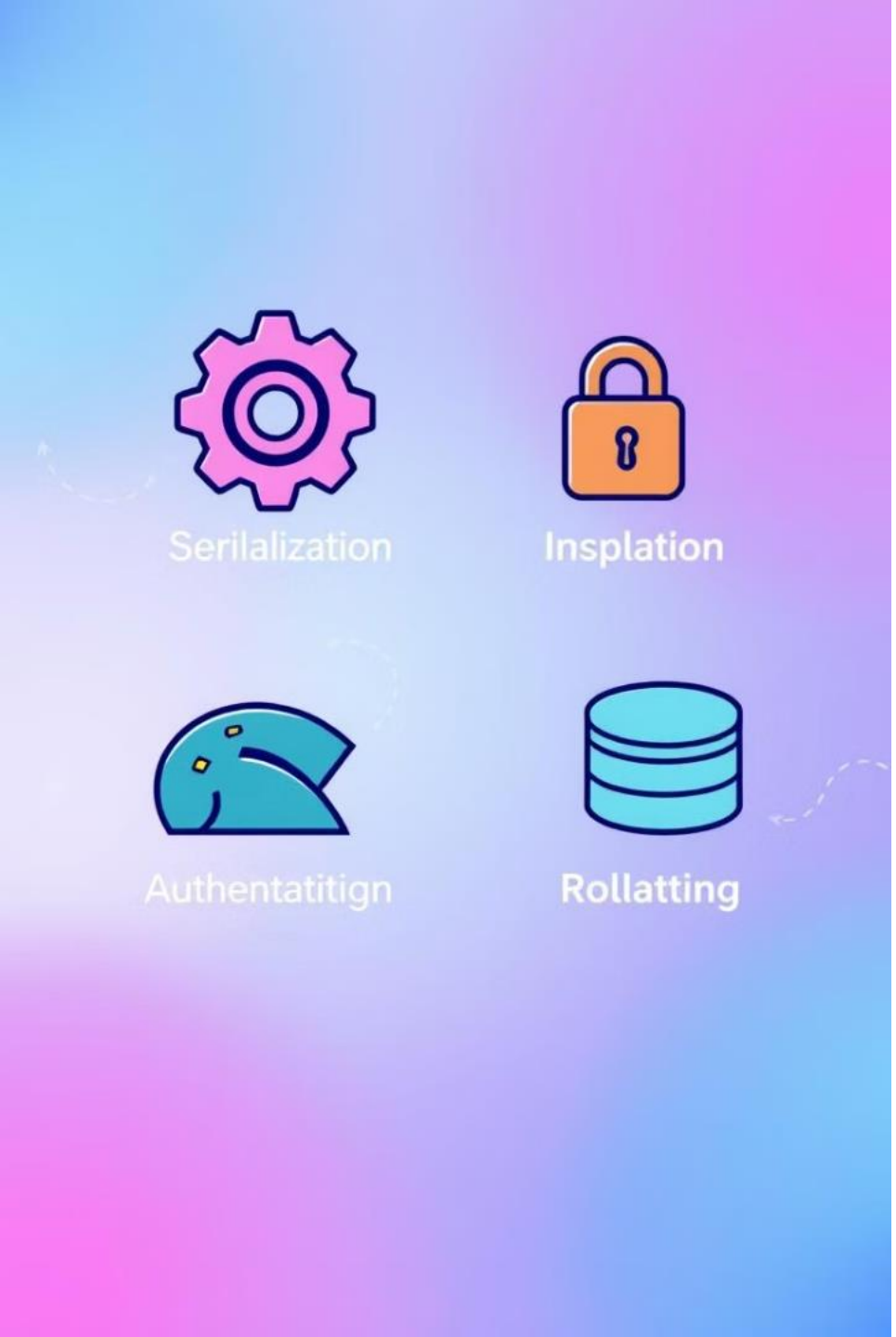
  - ✓ *BasicAuthentication* – insecure, for testing only
  - ✓ *SessionAuthentication* – same session context as website, powers DRF visualizer
  - ✓ *TokenAuthentication* – secure, recommended default
  - ✓ *RemoteUserAuthentication* – intranet sites, rarely used





Basic Architecture





Key Features of Django REST Framework

1 **Serialization**

- Powerful framework for converting complex Python objects to JSON, XML, etc.
- It supports both ORM and non-ORM data sources.

2 **Authentication & Permissions**

- Built-in support for various auth methods and fine-grained access control

3 **Viewsets & Routers**

- Classes to combine API logic and auto-generate URL routes.

4 **Request/Response Handling**

- Parsers and renderers to simplify data handling in multiple formats.



API Design Best Practices

Use RESTful Principles

1

Follow REST conventions for resource naming, HTTP methods, and URL structures.

Implement Versioning

2

Include version numbers in URLs or headers for future-proofing.

Utilize Serializers

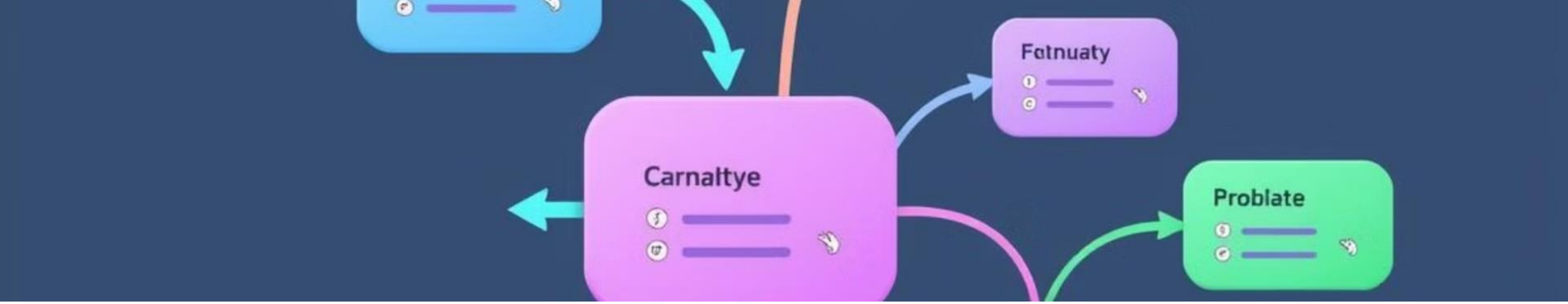
3

Use DRF serializers for efficient data validation and transformation.

Proper HTTP Status Codes

4

Return appropriate codes to indicate request outcomes.



Designing Models for API Data Representation

Simplicity

Keep models simple, representing single entities or concepts

Proper Field Definition

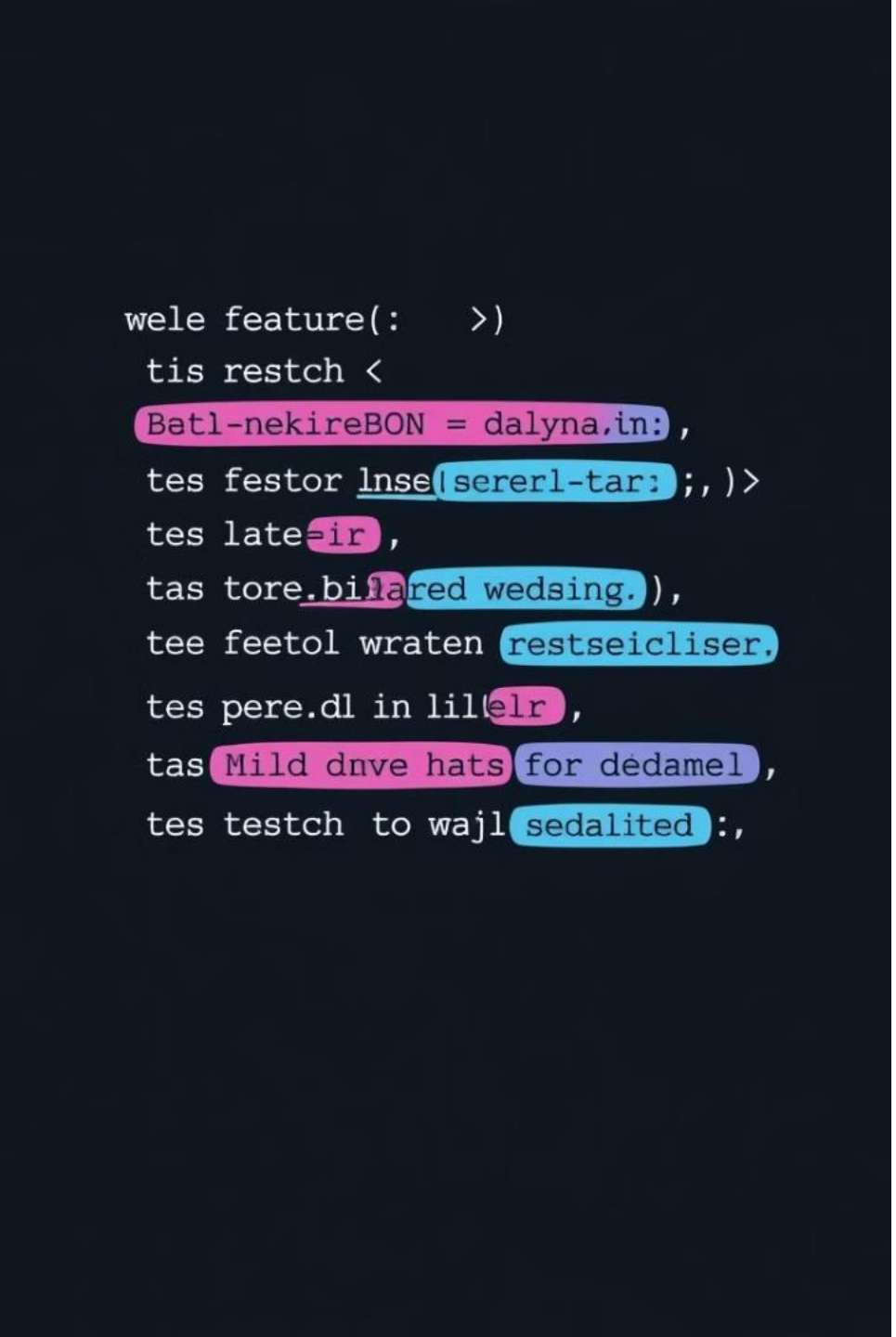
Choose appropriate field types and use constraints/validators

Establish Relationships

Define accurate connections between models using appropriate fields

Normalization

Follow database normalization principles to maintain data integrity



Creating Django Serializers

1

Import Modules

from rest_framework import serializers

2

Create Serializer Class

class YourModelSerializer(serializers.ModelSerializer):

3

Define Meta Class

```
class Meta:
        model = YourModel
        fields = '__all__'
```

4

Customize Fields

Add custom fields or methods as needed

Developing API Views: Class-Based Views

Advantages

- Structured and reusable
- Built-in functionality
- Easier to handle complex logic
- Supports inheritance and mixins

Example

```
class YourAPIView(APIView):
        def get(self, request): # Logic for GET request
                return Response(data)
```

Developing API Views: Function-Based Views

Advantages

- Simple and straightforward
- Suitable for small, focused views
- Easy to understand and maintain

Example

```
@api_view(['GET’])
def your_api_view(request): # Logic for GET request
        return Response(data)
```



Testing API Endpoints: Popular Tools

Postman

User-friendly interface for creating, sending, and managing API requests

Insomnia

Clean interface with advanced customization options

Swagger

UI for command interface

DRF UI

Web-based UI to facilitate API security and nagivation

Newman

Command-line tool for running Postman collections



API Hands_on
(On top of the User Profile, Add Seed and SeedCategories)

Seed and Categories Solution – Example -1

- Project is **SeedWeb**
- Application **myseedApp**

Make sure your virtual environment is active

- **pip install djangorestframework**
- **pip install djangorestframework-simplejwt** *# for token authentication*

Update the PROJECT SET UP Accordingly



- Add **REST_FRAMEWORK = {}**
- Add something in **INSTALLED_APPS = []**

- Add more **models** in models.py
- Add **views_api.py**
- Add **urls_api.py**
- Add **serializers.py**

Create MODELS

In myseedApp/models.py

```python
from django.db import models
from django.contrib.auth.models import User

# Create your models here.
class Profile(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE)
    bio = models.TextField(max_length=500, blank=True)
    location = models.CharField(max_length=30, blank=True)
    birth_date = models.DateField(null=True, blank=True)

class SeedCategory(models.Model):
    name = models.CharField(max_length=50)

    def __str__(self):
        return self.name

class Seed(models.Model):
    name = models.CharField(max_length=100)
    category = models.ForeignKey(SeedCategory, related_name='seeds', on_delete=models.CASCADE)
    quantity = models.IntegerField(default=0)

    def __str__(self):
        return self.name
```

Create MODELS

In myseedApp/models.py

- python manage.py makemigrations
- python manage.py migrate

Create Serializers

In myseedApp create a file called serializers.py

```python
from rest_framework import serializers
from django.contrib.auth.models import User
from .models import SeedCategory, Seed

class UserSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields = ['id', 'username', 'email']

class SeedCategorySerializer(serializers.ModelSerializer):
    class Meta:
        model = SeedCategory
        fields = ['id', 'name']

class SeedSerializer(serializers.ModelSerializer):

    category = SeedCategorySerializer(read_only = True)
    category_id = serializers.PrimaryKeyRelatedField(
        queryset=SeedCategory.objects.all(),
        source='category', write_only=True)

    class Meta:
        model = Seed
        fields = ['id', 'name', 'quantity', 'category', 'category_id']

```

Create views_api

In myseedApp create a file called views_api.py

```python
from rest_framework import viewsets
from django.contrib.auth.models import User
from rest_framework.permissions import AllowAny
from .models import SeedCategory, Seed
from .serializers import UserSerializer, SeedCategorySerializer, SeedSerializer

# User ViewSet (Public)
class UserViewSet(viewsets.ReadOnlyModelViewSet):
    queryset = User.objects.all()
    serializer_class = UserSerializer
    permission_classes = [AllowAny] # No auth for testing

class SeedCategoryViewSet(viewsets.ModelViewSet):
    queryset = SeedCategory.objects.all()
    serializer_class = SeedCategorySerializer
    #permission_classes = [AllowAny]

class SeedViewSet(viewsets.ModelViewSet):
    queryset = Seed.objects.all()
    serializer_class = SeedSerializer
```

Create urls_api

In myseedApp create a file called urls_api.py

```python
from django.urls import path, include
from rest_framework.routers import DefaultRouter
from .views_api import UserViewSet, SeedCategoryViewSet, SeedViewSet
from rest_framework_simplejwt.views import TokenObtainPairView, TokenRefreshView

router = DefaultRouter()
router.register(r'users', UserViewSet)
router.register(r'categories', SeedCategoryViewSet)
router.register(r'seeds', SeedViewSet)

urlpatterns = [
    path('api/', include(router.urls)),
    path('api/token/', TokenObtainPairView.as_view(), name ='token_obtain_pair'),
    path('api/token/refresh/', TokenObtainPairView.as_view(), name='token_refresh'),
    path('api-auth/', include('rest_framework.urls'))
]
```

Project urls

In SeedWeb allocated a file called urls.py

```python
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('myseedApp.urls')),
    path('', include('myseedApp.urls_api')),
]
```

SETTINGS

In SeedWeb allocate a file called settings.py

```python
INSTALLED_APPS = [
    'myseedApp',
    'rest_framework', #Added to render the API Test on the Web
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
]

# DRF Configuration
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': (
        'rest_framework_simplejwt.authentication.JWTAuthentication',
        'rest_framework.authentication.SessionAuthentication',  # required for web UI login/logout links
    ),
    'DEFAULT_PERMISSION_CLASSES': (
        'rest_framework.permissions.IsAuthenticated',
    ),
    'DEFAULT_VERSIONING_CLASS': 'rest_framework.versioning.NamespaceVersioning', # Versioning
    'DEFAULT_VERSION': 'v1', # Default Version
}
```

ADMIN

In SeedWeb allocate a file called admin.py

Register Models and format them in the ADMIN interface

```python
from django.contrib import admin
from .models import Profile, SeedCategory, Seed


# Display them in tabular format in Admin
class SeedCategoryAdmin(admin.ModelAdmin):
    list_display = ('id', 'name') # Columns in admin list view

class SeedAdmin(admin.ModelAdmin):
    list_display = ('id', 'name', 'category', 'quantity') # columns
    list_filter = ('category',) # Sidebar filter
    search_fields = ('name',) # search box

class ProfileAdmin(admin.ModelAdmin):
    list_display = ('user', 'bio', 'location', 'birth_date')

#MODEL REGISTRATION and FORMATTING in Admin UI
admin.site.register(Seed, SeedAdmin)
admin.site.register(SeedCategory, SeedCategoryAdmin)
admin.site.register(Profile, ProfileAdmin)
```

Run Server

Testing with Postman/DRF Web UI

1. Public Access (No Auth)

GET http://127.0.0.1:8000/api/users/ → List all users (no token needed because AllowAny)
GET http://127.0.0.1:8000/api/categories/ → Requires authentication by default (try it without token → 401)

2. Authenticated Access (JWT Token)

POST http://127.0.0.1:8000/api/token/ with body:

Made with Gamma



Example -2 Building REST API with Django for Post and Comments with codes

- By following best practices in API design, model creation, and serialization, developers can build scalable and maintainable APIs.
- Utilizing appropriate views and thorough testing with tools like Postman ensures the reliability and functionality of your API endpoints.

Building a Post and Comments API with Django & DRF

https://www.django-rest-framework.org/#example

- Install necessary packages.
- Start a new project and app.
- **Code:**

```
1. pip install django
2. djangorestframework drf-yasg
3. django-admin startproject
4. blog_project
5. cd blog_project
6. django-admin startapp blog
```

- Settings:

```
INSTALLED_APPS = [
    'rest_framework',
    'blog',
    'drf_yasg',
]
```

Define Post and Comment Models

- Post model for creating blog posts.
- Comment model to store comments related to each post.
- Code:

```
# blog/models.py

from django.db import models

class Post(models.Model):
    title = models.CharField(max_length=100)
    content = models.TextField()
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.title

class Comment(models.Model):
    post = models.ForeignKey(Post, related_name='comments', on_delete=     models.CASCADE)
    content = models.TextField()
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"Comment on {self.post.title}"
```

Create Serializers for API Data Conversion

- \\Serializers handle data conversion for API requests and responses.
- **Code**:

```
# blog/serializers.py

from rest_framework import serializers
from .models import Post, Comment

class CommentSerializer(serializers.ModelSerializer):
    class Meta:
        model = Comment
        fields = ['id', 'content', 'created_at']

class PostSerializer(serializers.ModelSerializer):
    comments = CommentSerializer(many=True, read_only=True)

    class Meta:
        model = Post
        fields = ['id', 'title', 'content', 'created_at', 'comments']
```

Set Up Views for API Endpoints

- List, create, and retrieve posts and comments.
- Code:

```
# blog/views.py

from rest_framework import generics
from .models import Post, Comment
from .serializers import PostSerializer,
CommentSerializer

class PostListCreateView(generics.ListCreateAPIView):
    queryset = Post.objects.all()
    serializer_class = PostSerializer

class
PostDetailView(generics.RetrieveUpdateDestroyAPIView):
    queryset = Post.objects.all()
    serializer_class = PostSerializer

class
CommentListCreateView(generics.ListCreateAPIView):
    serializer_class = CommentSerializer

    def get_queryset(self):
        return
Comment.objects.filter(post_id=self.kwargs['post_id'])

    def perform_create(self, serializer):
        post =
Post.objects.get(id=self.kwargs['post_id'])
        serializer.save(post=post)
```

Define URLs for Posts and Comments

- Set up URL routing to access the views.
- Code:

```
# blog/urls.py

from django.urls import path
from .views import PostListCreateView, PostDetailView,
CommentListCreateView

urlpatterns = [
    path('posts/', PostListCreateView.as_view(),
name='post-list-create'),
    path('posts/<int:pk>/', PostDetailView.as_view(),
name='post-detail'),
    path('posts/<int:post_id>/comments/',
CommentListCreateView.as_view(), name='comment-list-
create'),
]
```

Configure Basic Authentication

- Secure the endpoints by adding authentication.
- Settings:

```
# blog_project/settings.py

REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': [

'rest_framework.authentication.BasicAuthentication',
    ],
    'DEFAULT_PERMISSION_CLASSES': [

'rest_framework.permissions.IsAuthenticated',
    ],
}
```

Add Swagger for API Documentation

Use Swagger (drf-yasg) for interactive API documentation.

- Code:

```
# blog_project/urls.py

from rest_framework import permissions
from drf_yasg.views import get_schema_view
from drf_yasg import openapi

schema_view = get_schema_view(
    openapi.Info(
        title="Blog API",
        default_version='v1',
        description="API documentation for Post
and Comment",
    ),
    public=True,
    permission_classes=(permissions.AllowAny,),
)

urlpatterns += [
    path('swagger/',
schema_view.with_ui('swagger', cache_timeout=0),
name='schema-swagger-ui'),
]
```

Testing the API Endpoints

Start	Start the server and use Swagger at `/swagger/`.
POST	Example endpoints: - **Create Post**: POST `/api/posts/` - **View Posts**: GET `/api/posts/` - **Add Comment**: POST `/api/posts/<post_id>/comments/`
Run	Run Server: python manage.py runserver

Summary

Project Setup: Installed Django, DRF, drf-yasg.

Defined models, serializers, and views.

Added authentication and Swagger for API testing.

Tested the API with Swagger.

Testing Django Post and Comment API

A Guide for Step-by-Step Testing Using Insomnia and Swagger

1. Setting Up Insomnia for API Testing

- Open Insomnia and create a new Workspace.
- Add a Folder for organizing requests (e.g., 'Blog API').
- Set up Basic Authentication with your username and password.
- Use Basic Auth in each request for automatic authentication.

2. Testing Endpoints – Create a Post

- Method: POST
- URL: http://127.0.0.1:8000/api/posts/
- Headers: Content-Type: application/json
- Body:

```
{
    "title": "First Post",
    "content": "This is the content of post."
}
```

- Expected Response: 201 Created

3. Testing Endpoints - List All Posts



METHOD: GET



URL: HTTP://127.0.0.1:8000/API/POSTS/



HEADERS: ACCEPT: APPLICATION/JSON



EXPECTED RESPONSE: 200 OK WITH A LIST OF POSTS.

4. Testing Endpoints - Retrieve a Single Post by ID

Method: GET

URL: http://127.0.0.1:8000/api/posts/<id>/

Expected Response:

- 200 OK if the post exists.
- 404 Not Found if the post ID is invalid.

5. Testing Endpoints - Add a Comment to a Post

Method: POST

URL: http://127.0.0.1:8000/api/posts/<post_id>/comments/

Headers: Content-Type: application/json

Body:

```
{
        "content": "This is a comment on the first post."
}
```

Expected Response: 201 Created with comment details.

6. Testing Endpoints - List All Comments for a Post

Method: GET

URL: http://127.0.0.1:8000/api/posts/<post_id>/comments/

Expected Response: 200 OK with a list of comments.

Library Solution – Example -3

- Project is Bookapi
- Application Books

- pip install django djangorestframework
- django-admin startproject bookapi
- python manage.py startapp books

Book and Author Model

bookapi > books > models.py > ...

```python
from django.db import models



class Author(models.Model):
    first_name = models.CharField(max_length=255)
    last_name = models.CharField(max_length=255)
    biography = models.TextField(blank=True, null=True)
    date_of_birth = models.DateField(blank=True, null=True)

    def __str__(self):
        return f"{self.first_name} {self.last_name}"


class Book(models.Model):
    title = models.CharField(max_length=255)
    author = models.ForeignKey(Author, on_delete=models.CASCADE, related_name='books')
    description = models.TextField()
    published_date = models.DateField()

    def __str__(self):
        return self.title

```

Serializer

bookapi > books > serializers.py > BookCreateUpdateSerializer > Meta

```python
from rest_framework import serializers
from .models import Author, Book


class AuthorSerializer(serializers.ModelSerializer):
    books = serializers.StringRelatedField(many=True, read_only=True)  # List book titles

    class Meta:
        model = Author
        fields = ['id', 'first_name', 'last_name', 'biography', 'date_of_birth', 'books']


class BookSerializer(serializers.ModelSerializer):
    author = serializers.PrimaryKeyRelatedField(queryset=Author.objects.all())  # Use ID for author

    class Meta:
        model = Book
        fields = ['id', 'title', 'author', 'description', 'published_date']

class BookCreateUpdateSerializer(serializers.ModelSerializer):
    """A separate serializer for Book Create/Update operations to use author ID instead of full author details."""
    class Meta:
        model = Book
        fields = '__all__'
```

Views

bookapi > books > views.py > AuthorViewSet

```python
from rest_framework import viewsets
from rest_framework.permissions import IsAuthenticated
from .models import Author, Book
from .serializers import AuthorSerializer, BookSerializer, BookCreateUpdateSerializer


class AuthorViewSet(viewsets.ModelViewSet):
    queryset = Author.objects.all()
    serializer_class = AuthorSerializer
    permission_classes = [IsAuthenticated]


class BookViewSet(viewsets.ModelViewSet):
    queryset = Book.objects.all()
    permission_classes = [IsAuthenticated]

    def get_serializer_class(self):
        if self.action in ['create', 'update', 'partial_update']:
            return BookCreateUpdateSerializer
        return BookSerializer

```

APP,
URLS

bookapi > books > urls.py > ...

```python
from django.urls import path, include
from rest_framework.routers import DefaultRouter
from .views import AuthorViewSet, BookViewSet

router = DefaultRouter()
router.register(r'authors', AuthorViewSet)
router.register(r'books', BookViewSet)

urlpatterns = [
    path('', include(router.urls)),
]

```

Project, URLS

bookapi > bookapi > urls.py > ...

```
    The `urlpatterns` list routes URLs to views. For more information please see:
        https://docs.djangoproject.com/en/5.1/topics/http/urls/
    Examples:
    Function views
        1. Add an import:  from my_app import views
        2. Add a URL to urlpatterns:  path('', views.home, name='home')
    Class-based views
        1. Add an import:  from other_app.views import Home
        2. Add a URL to urlpatterns:  path('', Home.as_view(), name='home')
    Including another URLconf
        1. Import the include() function: from django.urls import include, path
        2. Add a URL to urlpatterns:  path('blog/', include('blog.urls'))
    """
    from django.contrib import admin
    from django.urls import path, include

    urlpatterns = [
        path('admin/', admin.site.urls),
        path('api/', include('books.urls')),  # API URLs
    ]
```




```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',

    'rest_framework',
    'books',  # New app
]
```

Library Solution - Example

- Remember to run migrations for database creations

- python manage.py runserver

API Versioning

01

Take-home Assignment